

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина: Операционные среды и системное программирование

К защите допустить:

И.О. Заведующего кафедрой
информатики

_____ С. И. Сиротко

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту

на тему

**СПИСОК ПРОЦЕССОВ, ХАРАКТЕРИСТИКИ, УПРАВЛЕНИЕ
(АНАЛОГ TASKMANAGER WIN NT)**

БГУИР КП 1-40 04 01 009 ПЗ

Студент

Е. С. Кахновский

Руководитель

Н. Ю. Гриценко

Минск 2024

СОДЕРЖАНИЕ

Введение.....	5
1 Диспетчер задач Windows	7
1.1 Структура диспетчера задач Windows.....	7
1.2 История, версии и достоинства	14
1.3 Анализ функциональности, пользовательского интерфейса и уязвимостей	28
2 Платформа программного обеспечения.....	34
2.1 Структура и архитектура платформы	34
2.2 История, версии и достоинства	35
2.3 Обоснование выбора платформы	37
3 Теоретическое обоснование разработки программного продукта.....	38
3.1 Обоснование необходимости разработки.....	38
3.2 Технологии программирования, используемые для решения поставленных задач	40
4 Проектирование функциональных возможностей программы	43
4.1 Разработка API	43
4.2 Вспомогательные классы	47
4.3 Task Manager.....	49
5 Взаимодействие с программным продуктом	52
5.1 Общие возможности программы.....	52
5.2 Приложения	57
5.3 Процессы.....	59
5.4 Службы и пользователи	60
Заключение	62
Список литературных источников	63
Приложение А (обязательное) Листинг кода	65
Приложение Б (обязательное) Функциональная схема.....	75
Приложение В (обязательное) Блок схема алгоритма	76
Приложение Г (обязательное) Скриншоты работы программы	76
Приложение Д (обязательное) Ведомость документов.....	77

ВВЕДЕНИЕ

Операционные среды и системное программирование являются краеугольными камнями в индустрии информационных технологий, обеспечивая надежное и эффективное функционирование компьютерных систем. Они формируют основу для работы программного обеспечения и обеспечивают взаимодействие между аппаратными ресурсами и пользовательскими приложениями.

С развитием технологий и появлением новых требований к программному обеспечению, операционные среды сталкиваются с рядом вызовов, включая увеличение объема данных, рост сложности систем и повышение требований к безопасности.

Современная информационная эпоха характеризуется внушительным увеличением числа процессов, которые запускаются и исполняются на компьютерах и других устройствах.

В первую очередь, увеличение количества процессов связано с развитием сферы информационных технологий, включая различные области от бизнеса до развлечений.

Большое количество приложений и сервисов требуют множества процессов для своего функционирования. Пользовательские программы, такие как текстовые редакторы, веб-браузеры, игры и мультимедийные приложения, создают собственные процессы для выполнения своих задач.

Кроме того, современные операционные системы имеют множество служб и фоновых процессов, обеспечивающих различные функции, такие как обновления, антивирусная защита, обработка сетевых запросов и многое другое.

Эти процессы работают параллельно с пользовательскими приложениями и обеспечивают функциональность и безопасность операционной системы.

Важно отметить также рост числа сетевых процессов, связанных с обменом данными между компьютерами и другими устройствами. С развитием интернета вещей (IoT) и облачных вычислений, сетевая активность и количество процессов, обрабатывающих сетевые запросы, также значительно возрастает.

Системные задачи включают в себя процессы, необходимые для обеспечения работы операционной системы, такие как обновления, антивирусная защита, планирование задач и многое другое.

Сетевые процессы отвечают за обмен данными между компьютерами в

сети, включая передачу файлов, доступ к веб-сервисам, электронную почту и т.д.

Управление этими процессами играет решающую роль в обеспечении стабильной и эффективной работы информационных систем.

Мониторинг процессов позволяет отслеживать их состояние, использование ресурсов системы и выявлять потенциальные проблемы.

Оптимизация процессов направлена на повышение производительности, улучшение отклика системы и оптимизацию использования ресурсов, что в свою очередь способствует улучшению пользовательского опыта и снижению издержек на обслуживание информационной инфраструктуры.

Целью данного курсового проекта является исследование списка процессов, анализ их характеристик и разработка эффективных методов управления ими.

Особое внимание уделяется созданию аналога *Task Manager* для операционной системы *Windows* – инструмента, который позволит пользователям мониторить текущие процессы, анализировать их использование ресурсов и управлять ими для оптимизации работы системы.

Для достижения данной цели предусмотрены следующие задачи:

1 Изучение архитектуры операционной системы *Windows* и принципов работы процессов, приложений, служб в ней.

2 Анализ существующих методов и инструментов управления процессами, приложениями, службами и пользователями.

3 Разработка алгоритмов для мониторинга и управления процессами в операционной системе *Windows*.

4 Проектирование пользовательского интерфейса для аналога *Task Manager*.

5 Реализация и взаимодействие с программным продуктом, обеспечивающим функциональность аналога *Task Manager*.

Пояснительная записка оформлена в соответствии с СТП 01-2017 [1].

1 ДИСПЕТЧЕР ЗАДАЧ WINDOWS

1.1 Структура диспетчера задач Windows

Диспетчер задач Windows является встроенным инструментом операционной системы Windows, который предназначен для мониторинга и управления различными аспектами работы компьютера (рисунок 1.1).

Он предоставляет пользователям возможность получать информацию о запущенных приложениях, службах и процессах, а также контролировать их работу и использование системных ресурсов, таких как процессор, память и дисковое пространство.

В Диспетчере задач можно наблюдать список всех активных процессов, их названия и данные о потребляемых ресурсах, таких как процессор и оперативная память.

Также можно просматривать запущенные службы и их статус, а также осуществлять управление ими, например, останавливать или запускать службы по необходимости.

Этот инструмент полезен для мониторинга производительности компьютера, выявления проблемных процессов или служб, а также оптимизации использования ресурсов [2].

Диспетчер задач

ФайлПараметрыВид

ПроцессыПроизводительностьЖурнал приложенийАвтозагрузкаПользователиПодробностиСлужбы

Имя	ИД процесса	С...	Имя процесса	1% ЦП	56% Память	2% Диск	0% Сеть	0% GPU	Ядро GPU	Энергопотре...	Тенденция эн...
Приложения (6)											
> Проводник (2)	16436		explorer.exe	0%	44,5 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое	Очень низкое
> Диспетчер задач	29536		Taskmgr.exe	0,2%	31,3 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое	Очень низкое
> Opera Internet Browser (25)				0,1%	1 481,0 МБ	0,1 МБ/с	0 Мбит/с	0%	Графичес...	Очень низкое	Очень низкое
> MSYS2 terminal	26260		mintty.exe	0%	1,6 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое	Очень низкое
> Microsoft Word	30536		WINWORD.EXE	0,1%	78,8 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое	Очень низкое
> Microsoft Edge (7)				0,2%	56,6 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое	Очень низкое
Фоновые процессы (140)											
> Экран блокировки Windows п...				0%	0 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое	Очень низкое
Хост-процесс для задач Wind...	17656		taskhostw.exe	0%	2,6 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое	Очень низкое
Хост-процесс для задач Wind...	21580		taskhostw.exe	0%	0,2 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое	Очень низкое
Хост-процесс для задач Wind...	22552		taskhostw.exe	0%	0,1 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое	Очень низкое
Служба управления виртуаль...	2636		vmms.exe	0%	2,2 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое	Очень низкое

Меньше

Снять задачу

Рисунок 1.1 – Диспетчер задач Windows

Для того чтобы полностью понять структуру и возможности диспетчера задач Windows, важно иметь хорошее представление о базовых концепциях

операционной системы Windows, таких как приложения, процессы, потоки, волокна, ресурсы, службы.

Понимание этих основных понятий является ключом к успешному использованию диспетчера задач, так как этот инструмент предоставляет информацию и контроль над этими элементами операционной системы.

Приложения Windows представляют собой программные компоненты, разработанные для выполнения различных задач на операционной системе Windows.

Эти компоненты являются основой пользовательского опыта на Windows и охватывают широкий спектр функциональности, начиная от базовых утилит до сложных приложений для профессионального использования.

Интерфейс каждого приложения разрабатывается с учетом удобства взаимодействия с пользователем. Графический интерфейс (GUI) является наиболее распространенным и обеспечивает интуитивно понятное взаимодействие с помощью графических элементов, таких как кнопки, поля ввода, меню и диалоговые окна.

Текстовый интерфейс (CLI) предоставляет возможность взаимодействия с помощью текстовых команд, а командный интерфейс (CUI) позволяет пользователю выполнять команды с помощью командной строки.

Приложения Windows активно используют ресурсы компьютера для своей работы. Они могут использовать процессорное время для выполнения вычислений, оперативную память для временного хранения данных, дисковое пространство для сохранения файлов и сетевые ресурсы для обмена данными с удаленными серверами.

Каждое приложение имеет свои требования к ресурсам, которые могут варьироваться в зависимости от его сложности и функциональности.

Важной функцией приложений Windows является удовлетворение разнообразных потребностей пользователей. Они предоставляют средства для решения задач в различных областях, таких как работа, обучение, развлечения и коммуникации.

Пользователи могут использовать приложения для создания и редактирования документов (рисунок 1.2), обработки изображений и видео, просмотра веб-сайтов, игр, общения с друзьями и коллегами, а также многих других задач.

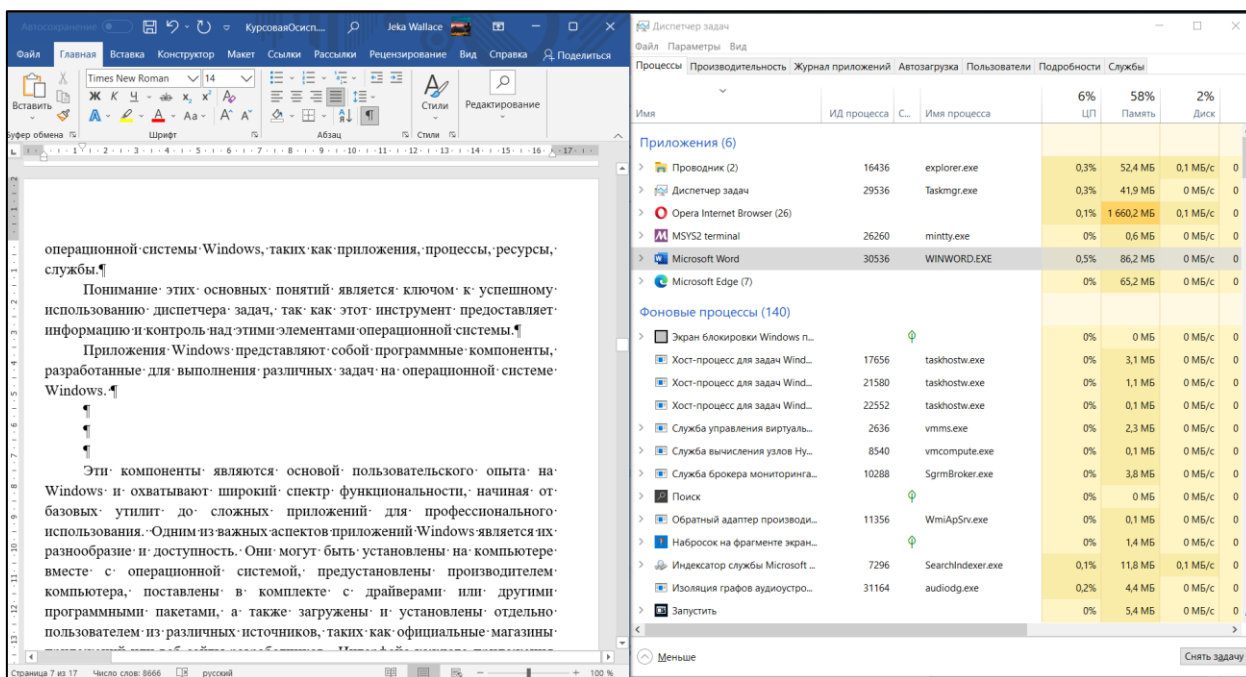


Рисунок 1.2 – Пример приложения MS Word

С помощью диспетчера задач пользователи могут завершать нежелательные процессы, оптимизировать использование ресурсов и улучшать производительность своего компьютера.

На первый взгляд может показаться, что процессы и приложения (в будущем программы) похожи, но в действительности между ними существуют принципиальные различия.

Программа — статическая последовательность команд, тогда как **процесс** — контейнер для набора ресурсов, используемых для выполнения программы (рисунок 1.3).

На верхнем уровне абстракции процесс Windows включает следующие компоненты [3]:

- 1 Закрытое виртуальное адресное пространство — множество адресов виртуальной памяти, которая может использоваться процессом;
- 2 Исполняемая программа, которая определяет первоначальный код и данные и отображается в виртуальное адресное пространство процесса;
- 3 Список открытых дескрипторов для различных системных ресурсов (семафоров, объектов синхронизации портов, файлов и т. д.), доступных для всех программных потоков в процессе;
- 4 Контекст безопасности — маркер доступа (access token), который идентифицирует пользователя, группы безопасности, привилегии, состояние виртуализации UAC (User Account Control), сеанс и ограниченное состояние учетной записи пользователя, связанное с процессом, а также идентификатор

контейнера приложения и связанная с ним информация изоляции;

5 Идентификатор процесса (PID) — уникальный идентификатор, который является частью идентификации клиента.

6 По меньшей мере один программный поток (thread).

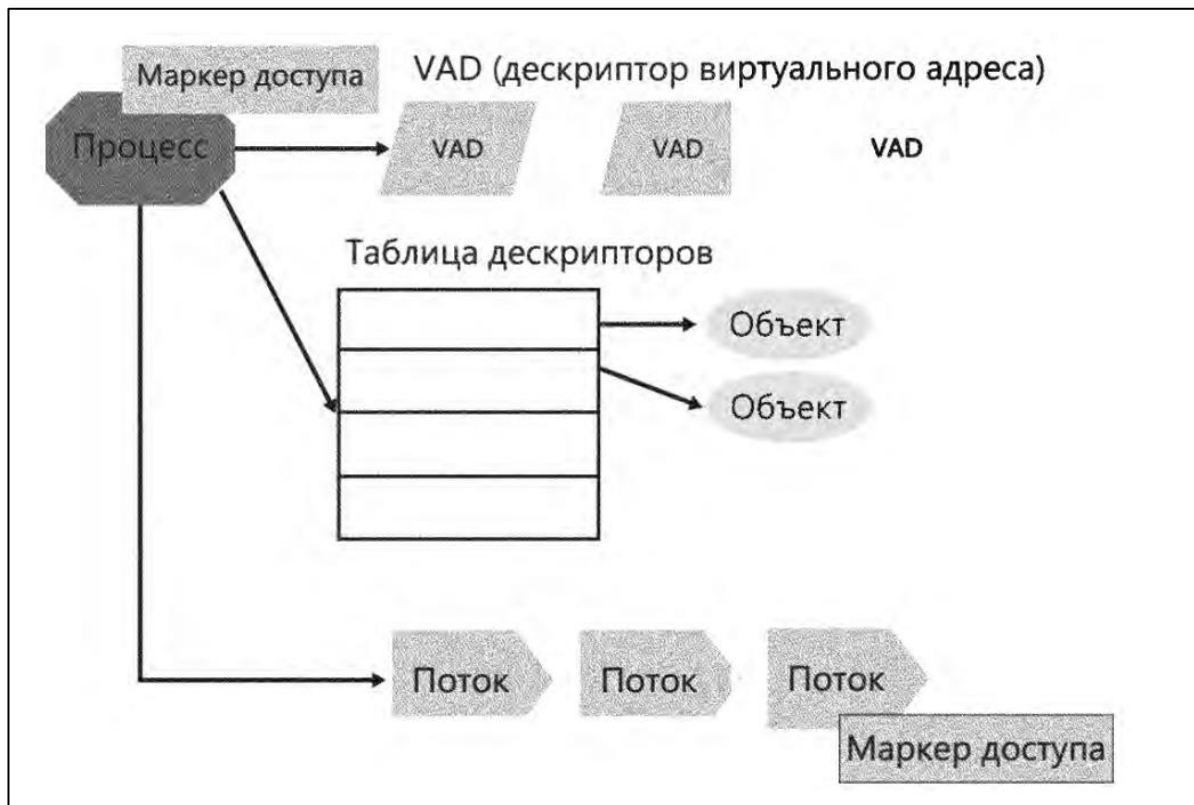


Рисунок 1.3 – Процесс и его ресурсы

Пустые процессы теоретически могут существовать, но особой пользы не принесут.

Для просмотра информации о процессах чаще всего используется диспетчер задач (Task Manager).

Программный поток (или просто поток) — последовательность команд внутри процесса, планируемая Windows для исполнения (рисунок 1.4). Без потоков программа процесса не сможет выполняться [4].



Рисунок 1.4 – Схема блока потока ядра Windows

Поток содержит следующие важнейшие компоненты:

1 Содержимое набора регистров CPU, представляющих состояние процессора.

2 Два стека: один для программного потока, который должен использоваться при выполнении в режиме ядра, другой — для выполнения в пользовательском режиме.

3 Закрытая область памяти, называемая локальной памятью потока команд (TLS, Thread-Local Storage); используется подсистемами, библиотеками времени выполнения и динамическими библиотеками DLL.

4 Уникальный идентификатор, называемый идентификатором потока (TID, Thread ID) является частью внутренней структуры идентификатора клиента (client ID). Идентификаторы процесса и потока генерируются из одного пространства имен, поэтому они никогда не перекрываются.

Кроме того, потоки иногда обладают собственным контекстом безопасности, который часто используется многопоточными серверными приложениями, олицетворяющими контекст безопасности обслуживаемых клиентов.

Регистры, стеки и закрытая область памяти называются контекстом потока. Так как эта информация зависит от архитектуры машины, на которой работает Windows, структура неизбежно привязывается к конкретной архитектуре.

Так как переключение выполнения между потоками требует участия планировщика ядра, эта операция может быть довольно затратной, особенно если два потока часто передают управление между собой.

В Windows реализованы два механизма для сокращения этих затрат: волокна (fibers) и планирование пользовательского режима (UMS, User Mode Scheduling).

Волокна позволяют приложению планировать свои потоки выполнения, а не полагаться на механизм приоритетного планирования, встроенный в Windows. Волокна также часто называют «облегченными потоками». В отношении планирования волокна невидимы для ядра, потому что они реализуются в пользовательском режиме в Kernel32.dll.

Ресурсы Windows включают в себя различные компоненты и аспекты, которые операционная система использует для обеспечения функциональности и производительности [5].

Эти ресурсы могут быть физическими, такими как процессорное время и оперативная память, или логическими, такими как файловые системы и сетевые соединения.

Основные ресурсы Windows:

1 Процессорное время (CPU) – это один из самых важных ресурсов операционной системы. Процессор обрабатывает инструкции программ и выполняет вычисления. Различные процессы и приложения могут конкурировать за доступ к процессору, и операционная система управляет их выполнением, приоритетами и распределением ресурсов.

2 Оперативная память (RAM) используется для временного хранения данных и инструкций, к которым часто обращается процессор. Операционная система управляет распределением оперативной памяти между различными процессами и приложениями, чтобы обеспечить эффективное выполнение программ.

3 Хранение данных на диске – еще один важный аспект работы операционной системы. Дисковое пространство используется для хранения операционной системы, приложений, файлов пользователя и других данных. Операционная система отслеживает доступное дисковое пространство и управляет файловой системой для организации и доступа к данным.

В целом, ресурсы Windows представляют собой ключевые компоненты операционной системы, которые обеспечивают ее функциональность, производительность и возможности для пользователей и приложений. Эффективное управление этими ресурсами важно для обеспечения стабильной работы системы и удовлетворения потребностей пользователей.

Службы Windows – это специальные программы, которые работают в фоновом режиме и обеспечивают функциональность и поддержку операционной системы Windows (рисунок 1.5). Они выполняют различные задачи, такие как управление ресурсами компьютера, обработка сетевых запросов, обновление программного обеспечения, обеспечение безопасности и другие.

Основные аспекты служб Windows [6]:

1 **Запуск и управление:** Службы запускаются при загрузке операционной системы и работают в фоновом режиме без участия пользователя. Они управляются служебным контроллером Windows (SCM), который отвечает за их запуск, остановку, приостановку и удаление. SCM также отслеживает зависимости между службами и контролирует их взаимодействие.

2 **Функциональность:** Службы выполняют широкий спектр функций, включая управление устройствами (например, драйверы устройств), обеспечение безопасности (например, антивирусные программы), сетевые сервисы (например, DHCP, DNS), планирование задач (например, резервное копирование), и многое другое.

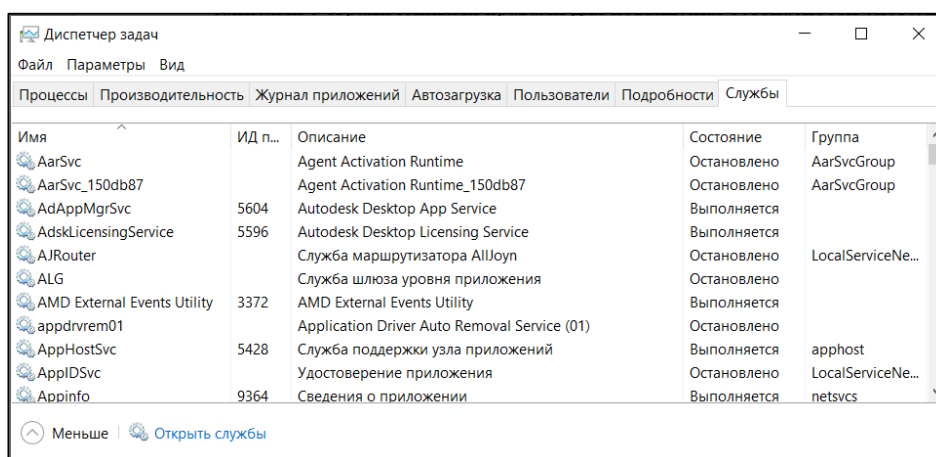


Рисунок 1.5 – Список служб в диспетчере задач Windows

В целом, службы Windows являются важной составной частью операционной системы Windows, обеспечивая ее функциональность, стабильность и безопасность. Они работают в фоновом режиме, обеспечивая поддержку и обработку различных задач, что делает их неотъемлемой частью работы компьютера под управлением Windows.

1.2 История, версии и достоинства

Исследование истории диспетчера задач в операционной системе Windows позволяет понять его эволюцию от начальных версий до современных релизов, а также выявить его достоинства и недостатки в каждой версии.

Диспетчер задач был представлен в его текущем виде начиная с Windows NT 4.0. Предыдущие версии Windows NT, а также Windows 3.x, включают в себя приложение «Список задач», способное перечислять текущие процессы и завершать их или создавать новый процесс.

Windows 3.x представляла собой семейство операционных систем, разработанных и выпущенных корпорацией Microsoft в начале 1990-х годов. В этот период диспетчер задач, или как он тогда еще назывался «Список задач», представлял собой базовый инструмент для управления процессами и ресурсами в системе.

Windows 3.0 была выпущена в 1990 году как первая успешная версия графического интерфейса для операционных систем MS-DOS. Она была обновлена до Windows 3.1 в 1992 году, и после этого вышла Windows 3.11, также известная как "Windows for Workgroups", которая добавила поддержку сети. В Windows 3.x отсутствовал отдельный приложение под названием "Диспетчер задач", как в более поздних версиях операционной системы. Вместо этого пользователи могли использовать комбинацию клавиш Ctrl + Alt + Delete для вызова окна, содержащего информацию о запущенных процессах и приложениях.

Диспетчер программ Windows (рисунок 1.6) предоставлял базовую функциональность для управления процессами и приложениями через простой интерфейс.

Пользователи могли завершать некорректно работающие приложения, что позволяло избежать зависаний и перезагрузок системы.

Окно Диспетчера программ Windows отображало список запущенных приложений и их состояние, что помогало пользователям контролировать активность системы.

Диспетчер программ Windows в Windows 3.x предоставлял только базовые возможности управления процессами и приложениями. Он не имел расширенных функций мониторинга и анализа ресурсов, доступных в более поздних версиях Windows.

В отличие от более поздних версий Windows, Диспетчер программ Windows в Windows 3.x не предоставлял дополнительных функций, таких как

мониторинг сетевой активности или анализ использования ресурсов.

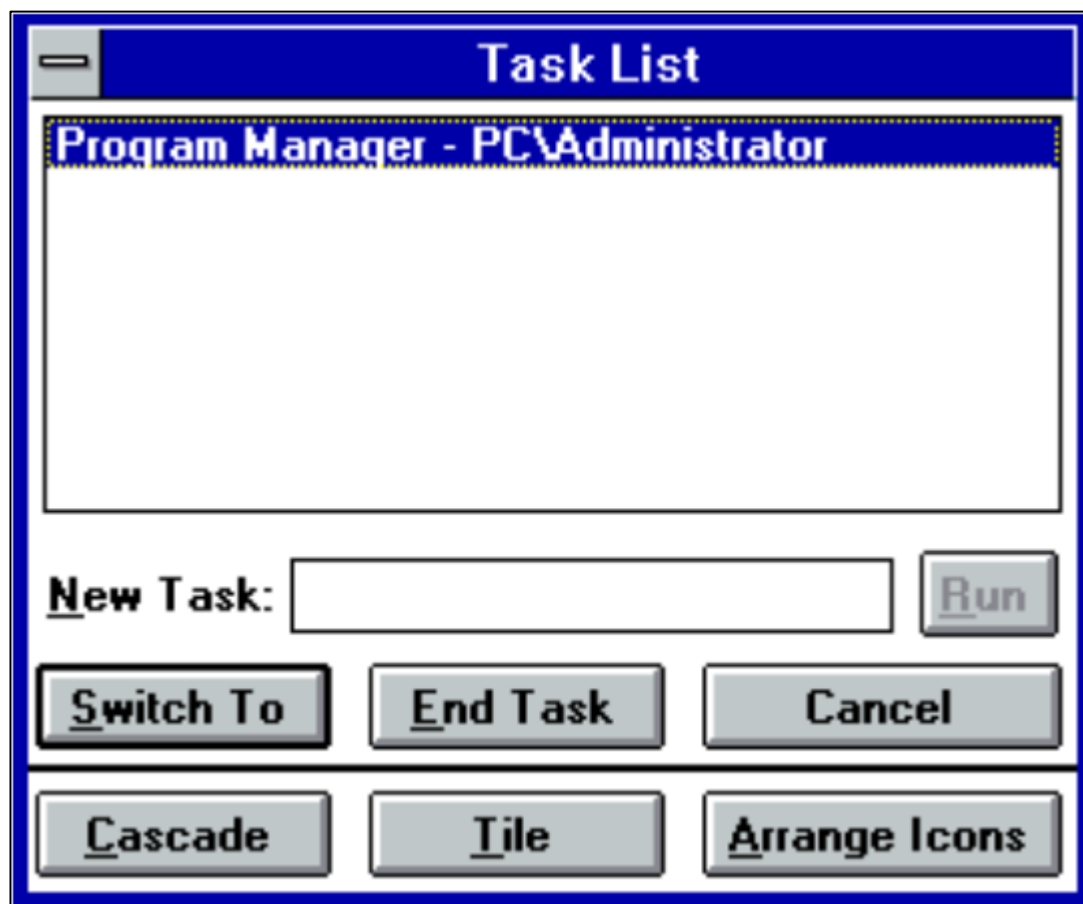


Рисунок 1.6 – Диспетчере задач Windows NT 3.1

В целом, диспетчер задач в Windows 3.x, хоть и имел базовый функционал, был важной частью операционной системы, предоставляя пользователям возможность управлять процессами и приложениями. Он являлся первым шагом к развитию более мощных инструментов управления в более поздних версиях Windows.

Windows 95 была выпущена Microsoft в августе 1995 года. Это было крупное событие в мире компьютерных технологий, поскольку Windows 95 представляла собой первую операционную систему Windows, которая не требовала предварительной установки MS-DOS. Это означало, что она могла работать самостоятельно, что существенно упростило процесс использования компьютера для пользователей [7].

Windows 95 представила пользовательский интерфейс, который стал основой для последующих версий операционных систем Windows. Он включал в себя стартовое меню, панель задач и область уведомлений, что значительно упростило навигацию по системе.

Одной из важных особенностей Windows 95 была поддержка Plug and Play, что позволяло устройствам автоматически распознаваться и устанавливаться без необходимости ручного вмешательства со стороны пользователя.

Windows 95 включала в себя встроенную поддержку Интернета и сетевых возможностей, что открыло новые возможности для пользователей в области связи и обмена данными.

Windows 95 предлагала значительные улучшения в области мультимедийных возможностей, включая поддержку звуковых карт и видео.

Диспетчер задач в Windows 95 (рисунок 1.7) представлял собой ключевой инструмент для управления процессами и ресурсами компьютера. В отличие от более ранних версий Windows, где пользователи использовали комбинацию клавиш для отображения списка запущенных программ, в Windows 95 Диспетчер задач был интегрирован в операционную систему и предоставлял расширенные возможности управления.

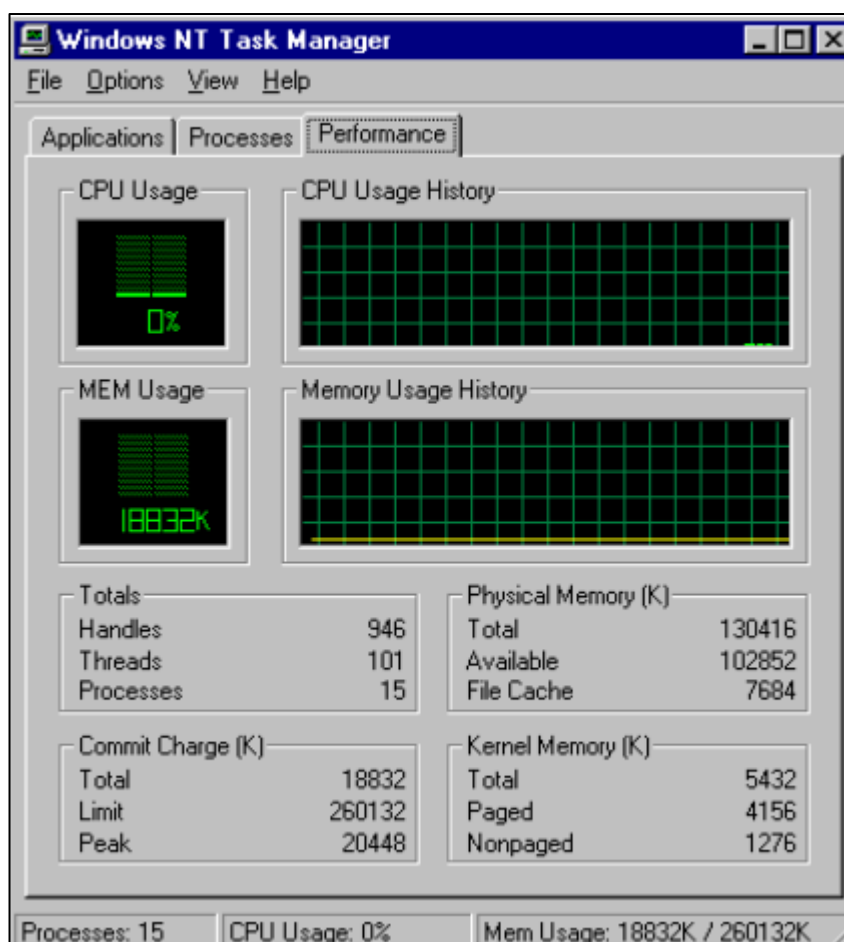


Рисунок 1.7 – Диспетчере задач Windows 95

Диспетчер задач в Windows 95 позволял пользователям просматривать список активных приложений и процессов. Завершать некорректно работающие приложения. Управлять ресурсами компьютера, такими как использование процессора и памяти.

Диспетчер задач был легко доступен и предоставлял интуитивно понятный интерфейс для управления процессами. Пользователи могли легко завершать некорректно работающие приложения, что помогало предотвратить зависание системы. Диспетчер задач в Windows 95 предоставлял базовый набор функций и не имел расширенных возможностей мониторинга и управления, доступных в более поздних версиях операционной системы. В целом, Windows 95 и ее Диспетчер задач были важным шагом в развитии операционных систем Windows, предоставляя пользователям новые возможности и упрощая работу с компьютером.

Windows 98 была разработана на основе Windows 95 и представляла собой следующий этап в развитии операционной системы. Ее выпуск был ответом на ожидания и запросы пользователей, а также учет новых технологических достижений.

Windows 98 включала ряд улучшений в производительности по сравнению с Windows 95. Оптимизации были внесены в загрузку системы, работу с памятью и файловой системой, что привело к более быстрой и стабильной работе.

Windows 98 предоставила более тесную интеграцию с Интернетом. Она включала в себя Internet Explorer 4.0, что упростило доступ пользователей к сети и веб-ресурсам.

В Windows 98 была значительно улучшена поддержка Plug and Play устройств, что сделало процесс установки и использования нового оборудования еще более простым и удобным для пользователя.

Windows 98 впервые включила в себя поддержку DVD и USB устройств, что открыло новые возможности для пользователей в области мультимедиа и хранения данных.

Диспетчер задач в Windows 98 (рисунок 1.8) продолжал выполнять свои основные функции управления процессами и ресурсами компьютера, подобно тому, как это было в Windows 95. Однако он также получил некоторые улучшения и дополнительные функции.

Диспетчер задач в Windows 98 имел более современный и интуитивно понятный интерфейс по сравнению с предыдущей версией. В Windows 98 были добавлены некоторые новые функции в Диспетчер задач, такие как дополнительная информация о процессах и ресурсах компьютера.

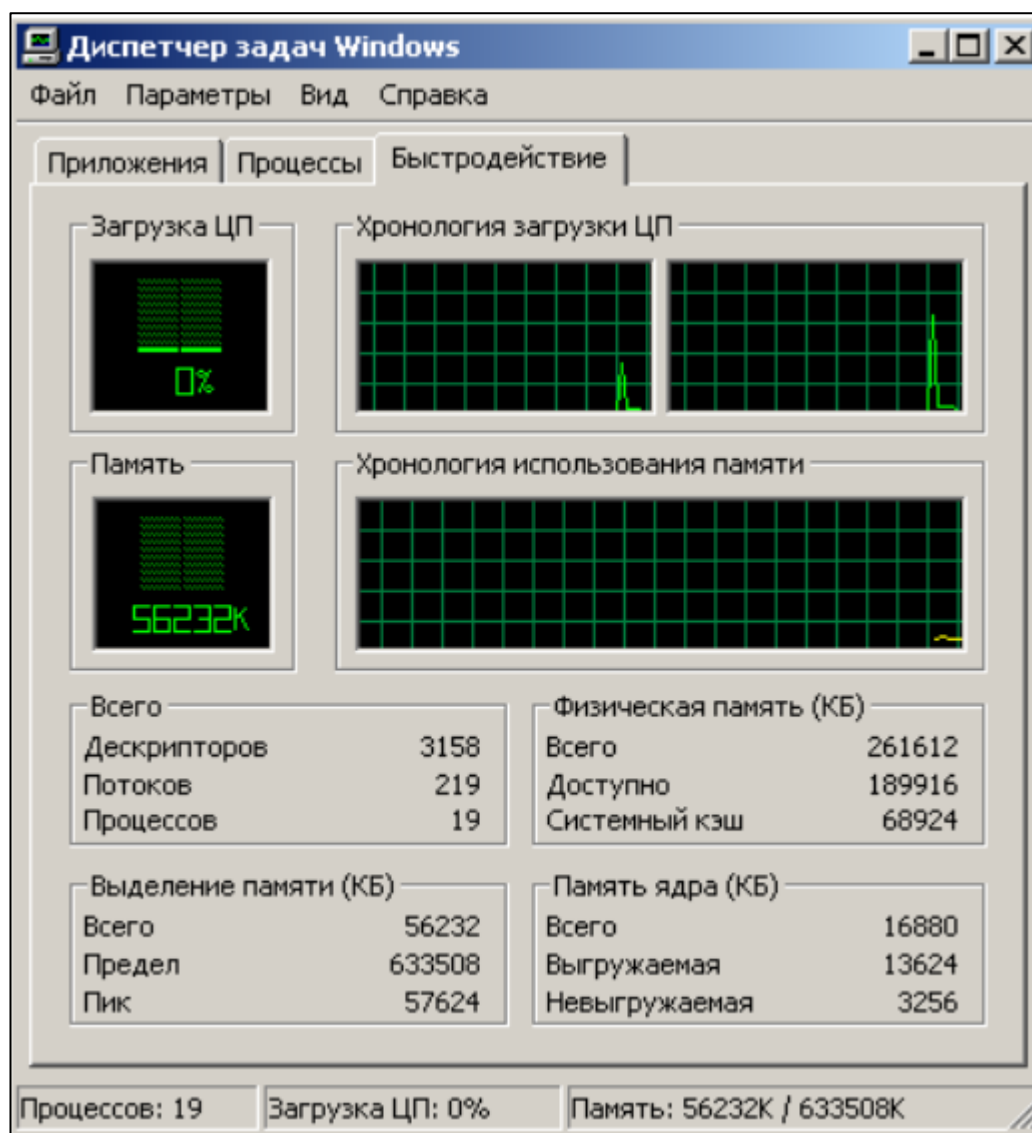


Рисунок 1.8 – Диспетчере задач Windows 98

Несмотря на улучшения, Диспетчер задач в Windows 98 все еще предоставлял базовый набор функций и не имел расширенных возможностей мониторинга и управления, доступных в более поздних версиях операционной системы. Windows 98 была важным шагом в развитии операционных систем Windows, предоставляя пользователям новые возможности и улучшенные функции. Диспетчер задач в Windows 98 продолжал служить важным инструментом для управления системой и ресурсами компьютера.

Windows XP была разработана на основе ядра Windows NT и являлась первой операционной системой Windows, объединяющей домашние и профессиональные версии в единый продукт. Это был важный шаг в унификации платформы Windows и создании универсальной операционной системы для всех категорий пользователей.

Windows XP представила новый пользовательский интерфейс, известный как "Luna", который включал в себя яркие цвета, прозрачные элементы и новые иконки. Это сделало операционную систему более современной и привлекательной для пользователей.

Windows XP значительно улучшила поддержку Plug and Play устройств, что сделало процесс установки и использования нового оборудования еще более простым и удобным для пользователя.

Windows XP включила в себя ряд новых функций для повышения безопасности системы, включая брандмауэр Windows, систему защиты данных и обновления безопасности через службу Windows Update.

Windows XP предоставила расширенную поддержку сети, включая поддержку беспроводных сетей и улучшенные возможности для работы в корпоративных средах

Диспетчер задач в Windows XP (рисунок 1.9) был существенно улучшен по сравнению с предыдущими версиями операционной системы [8].

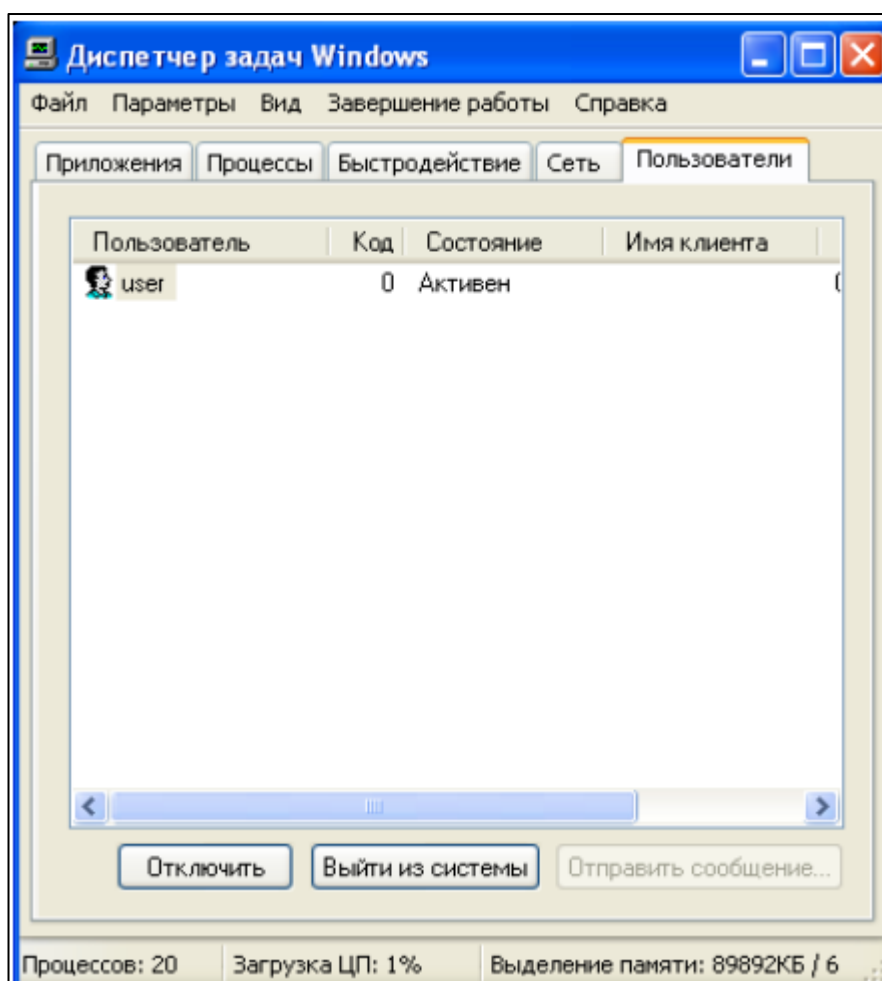


Рисунок 1.9 – Диспетчере задач Windows XP

Диспетчер задач в Windows XP предоставлял расширенный набор функций, включая подробную информацию о каждом запущенном процессе, а также возможность управления автозапуском приложений

Улучшения в Диспетчере задач помогали оптимизировать использование ресурсов компьютера и повышать производительность системы.

Несмотря на улучшения, Диспетчер задач в Windows XP все еще предоставлял ограниченные возможности мониторинга и управления, по сравнению с более современными версиями операционной системы.

Windows XP была важной версией операционной системы Windows, предоставившей пользователям новые возможности и улучшенный опыт использования компьютера. Диспетчер задач в Windows XP продолжал служить важным инструментом для управления процессами и ресурсами компьютера.

Windows 7, выпущенная компанией Microsoft в октябре 2009 года, стала одной из наиболее популярных и успешных версий операционной системы Windows. Этот релиз внес множество улучшений и новых функций, которые сделали его привлекательным как для домашних, так и для корпоративных пользователей.

Windows 7 была разработана как преемница Windows Vista, которая не получила широкой популярности из-за ряда проблем и недостатков. Windows 7 стремилась исправить эти недочеты и предложить более стабильную и производительную операционную систему для пользователей.

Windows 7 представила новый пользовательский интерфейс Aero, который включал в себя прозрачные окна, улучшенные анимации и возможность управления окнами с помощью Aero Shake и Aero Snap.

Windows 7 была значительно более стабильной и производительной по сравнению с Windows Vista. Улучшения в ядре операционной системы и оптимизация кода помогли сделать Windows 7 более отзывчивой и эффективной.

Windows 7 представила новый концепт библиотек, позволяющий пользователям организовать свои файлы и документы более удобно. Кроме того, в Windows 7 была интегрирована поддержка облачных служб, что позволяло пользователям легко обмениваться файлами через Интернет.

Windows 7 включала ряд новых функций для повышения безопасности системы, включая Управление учетными записями пользователей (UAC), антивирусную защиту Windows Defender и средства шифрования диска

BitLocker.

Диспетчер задач в Windows 7 (рисунок 1.10) предоставляет подробную информацию о каждом запущенном процессе, включая его имя, PID, потребление ресурсов и другие характеристики. Это позволяет пользователям легко отслеживать и управлять процессами.

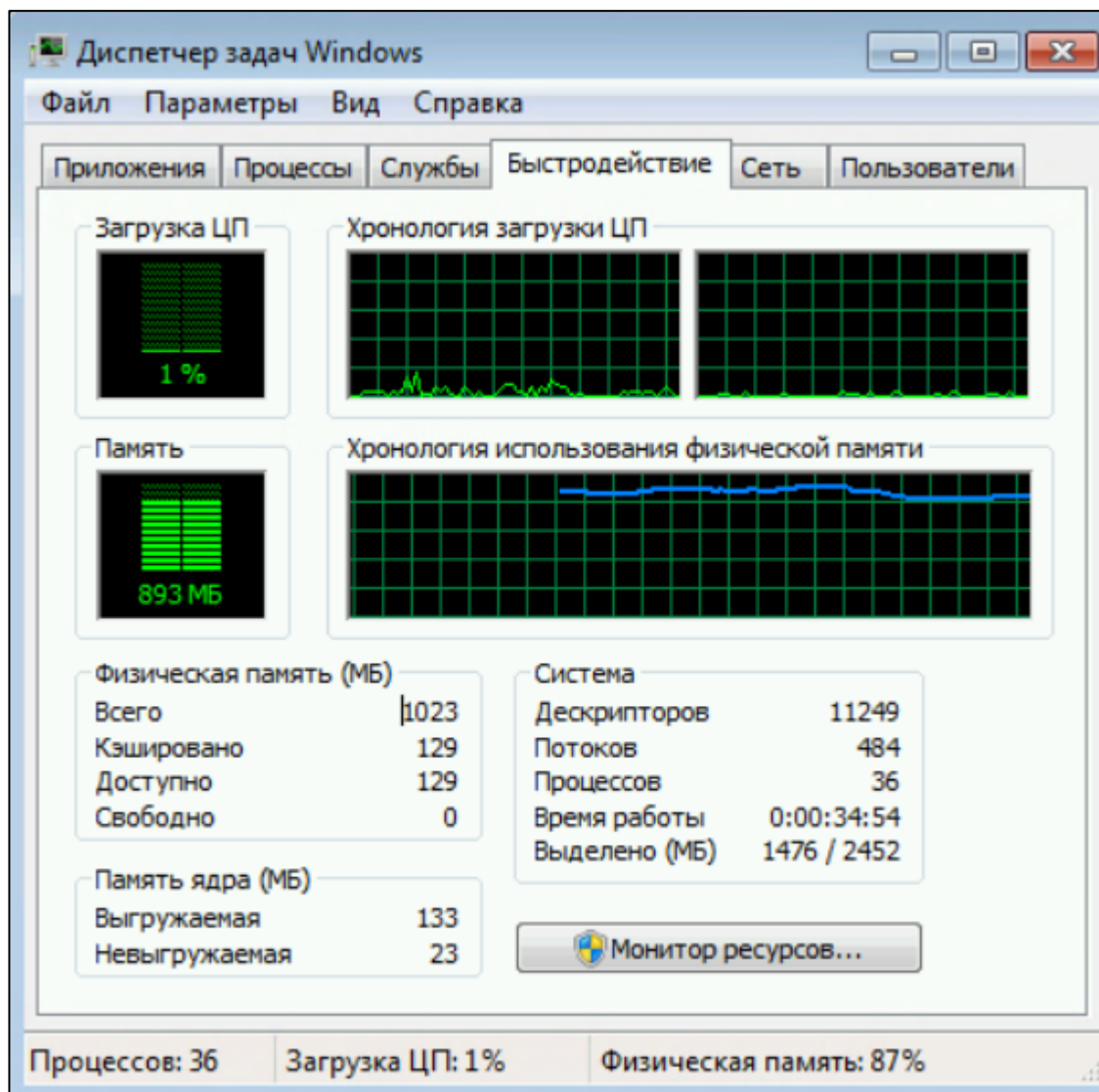


Рисунок 1.10 – Диспетчере задач Windows 7

Пользователи могут легко завершать некорректно работающие приложения или процессы через Диспетчер задач. Это позволяет быстро реагировать на замороженные приложения или избавляться от ненужных процессов, которые могут негативно влиять на производительность системы.

Встроенные инструменты мониторинга ресурсов позволяют пользователям отслеживать использование процессора, памяти, дискового пространства и сети. Это полезно для определения причин загрузки системы и

оптимизации ее работы.

Диспетчер задач в Windows 7 также позволяет управлять автозапуском приложений. Пользователи могут просматривать список программ, запускаемых при загрузке системы, и отключать ненужные или нежелательные приложения, что помогает ускорить загрузку компьютера.

Несмотря на улучшения, Диспетчер задач в Windows 7 все еще имеет ограниченные возможности диагностики и анализа процессов в сравнении с более современными инструментами. Например, он не предоставляет подробной информации о взаимосвязях между процессами или об их воздействии на производительность системы.

В Диспетчере задач в Windows 7 отсутствуют некоторые расширенные инструменты анализа производительности, которые могут быть полезны для опытных пользователей или системных администраторов. Например, он не предоставляет возможности анализа использования ресурсов на длительных временных промежутках или мониторинга системных событий.

Некоторые функции Диспетчера задач могут быть недостаточно интуитивными для неопытных пользователей. Например, завершение процессов или управление автозапуском приложений может потребовать определенных знаний о работе операционной системы.

Windows 7 стала одной из самых популярных версий операционной системы Windows благодаря своей стабильности, производительности и удобству использования. Диспетчер задач в Windows 7 продолжал служить важным инструментом для управления процессами и ресурсами компьютера.

Windows 8, выпущенная в октябре 2012 года, была следующей крупной версией операционной системы Windows после Windows 7. Этот релиз внес ряд изменений в пользовательский интерфейс и функциональность, направленных на адаптацию к современным требованиям и устройствам с сенсорным экраном [9].

Windows 8 была разработана как ответ на изменяющиеся потребности пользователей и смену трендов в компьютерных технологиях. Основной упор был сделан на поддержку сенсорных устройств, таких как планшеты и гибридные устройства, а также на улучшение интеграции с облачными сервисами.

Windows 8 представила новый пользовательский интерфейс под названием "Модерн" (или "Metro"), который был оптимизирован для сенсорных устройств. Этот интерфейс включал плитки, предоставляющие быстрый доступ к приложениям и информации.

Windows 8 улучшила интеграцию с облачными сервисами, такими как

OneDrive (ранее известный как SkyDrive), позволяя пользователям легко сохранять и синхронизировать свои файлы и настройки между различными устройствами.

Windows 8 предоставила возможность запуска приложений с главного экрана с помощью плиток, что сделало процесс запуска программ быстрее и удобнее. Также были введены новые жесты и команды для управления приложениями с сенсорных экранов.

Диспетчер задач в Windows 8 (рисунок 1.11) был модернизирован и адаптирован к новому пользовательскому интерфейсу "Модерн". Он сохранил основные функции предыдущих версий, но также включил некоторые новые возможности и улучшения.

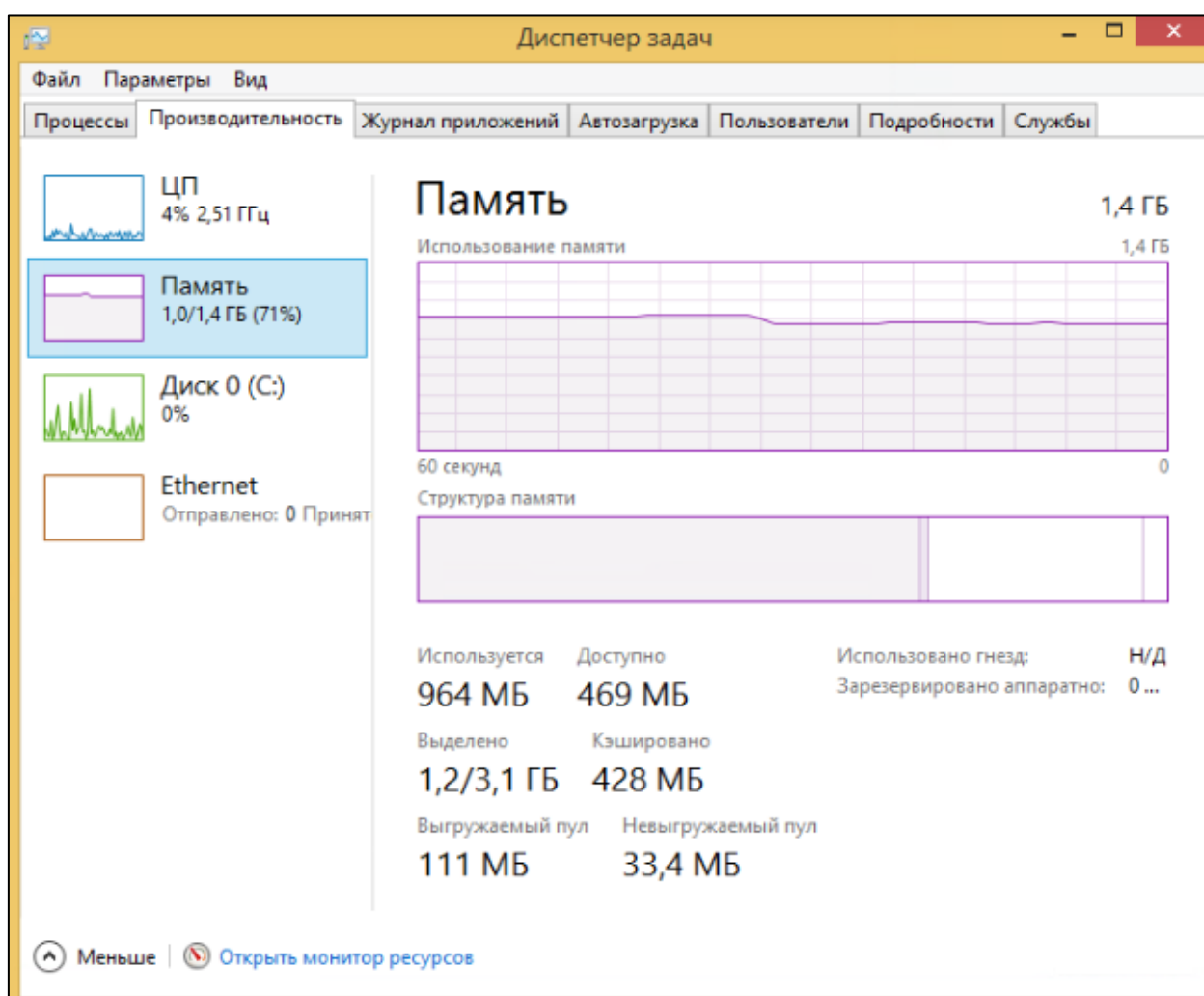


Рисунок 1.11 – Диспетчере задач Windows 8

Диспетчер задач был адаптирован к новому пользовательскому интерфейсу "Модерн", что обеспечило единый и удобный опыт использования для пользователей.

Диспетчер задач в Windows 8 предоставил расширенные возможности мониторинга процессов и ресурсов компьютера, что позволило пользователям более эффективно управлять работой своей системы.

Для некоторых пользователей новый интерфейс "Модерн" и измененный Диспетчер задач могли оказаться сложными для адаптации из-за отличий от предыдущих версий Windows

Несмотря на улучшения, Диспетчер задач в Windows 8 все еще имел ограниченные возможности диагностики и анализа процессов в сравнении с более современными инструментами.

Windows 8 принесла ряд новых функций и улучшений, в том числе для Диспетчера задач, который стал более удобным и функциональным инструментом для управления процессами и ресурсами компьютера.

Windows 10, выпущенная в июле 2015 года, стала одной из наиболее значимых версий операционной системы Windows, объединяя в себе лучшие аспекты предыдущих версий и внедряя ряд инноваций. Она призвана обеспечить универсальный опыт использования на различных типах устройств, включая настольные компьютеры, ноутбуки, планшеты и смартфоны.

Windows 10 была разработана компанией Microsoft в ответ на отзывы пользователей и с учетом последних тенденций в компьютерной индустрии.

Основными целями были улучшение пользовательского интерфейса, повышение производительности и расширение возможностей для разработчиков приложений.

Windows 10 внедрила концепцию универсального пользовательского интерфейса, который одинаково хорошо работает на различных типах устройств. Это включает в себя возможность запуска одних и тех же приложений на настольных компьютерах, планшетах и смартфонах.

Windows 10 внедрила модель непрерывных обновлений, что означает, что пользователи получают новые функции и исправления непосредственно во время их выпуска. Это помогает обеспечить актуальность и безопасность операционной системы.

Поддержка облачных сервисов, таких как OneDrive, в Windows 10 позволяет пользователям хранить и синхронизировать свои данные между различными устройствами и иметь к ним доступ из любой точки мира.

Диспетчер задач в Windows 10 получил несколько улучшений по сравнению с предыдущими версиями операционной системы. Он стал еще более удобным и функциональным инструментом для мониторинга и управления процессами и ресурсами компьютера [9].

Диспетчер задач в Windows 10 (рисунок 1.12) предоставляет более подробную информацию о процессах и ресурсах компьютера, что позволяет пользователям более эффективно управлять работой системы.

Имя	ИД процесса	С...	Имя процесса	9% ЦП	70% Память	2% Диск	2% Сеть	21% GPU
Приложения (7)								
> Проводник	9744		explorer.exe	0,1%	43,4 МБ	0 МБ/с	0 Мбит/с	0%
> Диспетчер задач	5328		Taskmgr.exe	4,1%	39,6 МБ	0,1 МБ/с	0 Мбит/с	0%
> Opera Internet Browser (23)				0,4%	985,4 МБ	0 МБ/с	0 Мбит/с	0%
> MSYS2 terminal	12292		mintty.exe	0%	4,2 МБ	0 МБ/с	0 Мбит/с	0%
> Microsoft Word	22436		WINWORD.EXE	0%	102,3 МБ	0 МБ/с	0 Мбит/с	0%
> Microsoft Visual Studio 2022 (14)				0%	1 222,7 МБ	0 МБ/с	0 Мбит/с	0%
> Discord (32 бита) (3)				0,9%	348,7 МБ	0,1 МБ/с	2,3 Мбит/с	20,8%
Фоновые процессы (141)								
> Экран блокировки Windows п...				0%	0 МБ	0 МБ/с	0 Мбит/с	0%
Хост-процесс для задач Wind...	10940		taskhostw.exe	0%	2,1 МБ	0 МБ/с	0 Мбит/с	0%

Рисунок 1.12 – Диспетчере задач Windows 10

Диспетчер задач в Windows 10 получил обновленный пользовательский интерфейс, который делает его более интуитивно понятным и удобным в использовании.

Несмотря на улучшения, Диспетчер задач в Windows 10 все еще может иметь ограниченные возможности для диагностики и анализа процессов в сравнении с более продвинутыми инструментами.

В сравнении с некоторыми сторонними инструментами мониторинга и управления, Диспетчер задач в Windows 10 может оставаться недостаточно функциональным для некоторых пользователей, особенно для профессионалов в области информационных технологий.

Windows 10 принесла множество улучшений и нововведений, включая в области управления ресурсами и мониторинга процессов через Диспетчер задач, что сделало этот инструмент более полезным и эффективным для пользователей операционной системы Windows.

Windows 11, представленная в июне 2021 года, является последней версией операционной системы Windows, разработанной компанией

Microsoft. Этот релиз внес ряд изменений в пользовательский интерфейс, функциональность и безопасность, нацеленных на повышение производительности и удобства использования для пользователей.

Windows 11 была анонсирована как следующая эволюционная ступень для Windows 10, призванная предложить улучшенный опыт использования настольных и мобильных устройств. Основной целью было создание более современного и адаптивного интерфейса, а также улучшение интеграции с облачными сервисами и безопасности операционной системы.

Windows 11 внедрила обновленный и более современный пользовательский интерфейс, который включает в себя централизованное меню Пуск, новые анимации и эффекты, а также улучшенные возможности настройки.

Поддержка облачных сервисов была улучшена в Windows 11, что позволяет пользователям легко сохранять, синхронизировать и обмениваться файлами и настройками между различными устройствами.

Windows 11 представила ряд новых функций для повышения безопасности операционной системы, включая улучшенную защиту от вредоносного программного обеспечения, расширенные инструменты аутентификации и защиту данных.

Диспетчер задач в Windows 11 получил ряд обновлений и улучшений по сравнению с предыдущими версиями операционной системы. Он стал еще более интуитивным и функциональным инструментом для мониторинга и управления процессами и ресурсами компьютера.

Улучшенный пользовательский интерфейс: Диспетчер задач в Windows 11 получил обновленный интерфейс, который делает его более интуитивно понятным и удобным в использовании.

В Windows 11 были внедрены дополнительные функции мониторинга процессов и ресурсов компьютера, что позволяет пользователям более эффективно управлять работой своей системы.

В силу внесенных изменений в операционную систему, некоторые сторонние приложения и инструменты могут испытывать проблемы с совместимостью с новым Диспетчером задач в Windows 11.

Как и в случае с любым новым интерфейсом, пользователи могут столкнуться с необходимостью адаптации к изменениям в Диспетчере задач Windows 11.

Windows 11 представляет собой значительный шаг вперед в развитии операционной системы Windows, предлагая обновленный пользовательский интерфейс, улучшенную интеграцию с облачными сервисами и повышенную

безопасность.

Диспетчер задач в Windows 11 (рисунок 1.13) также получил ряд обновлений и улучшений, что делает его более эффективным и удобным инструментом для мониторинга и управления процессами и ресурсами компьютера [10].

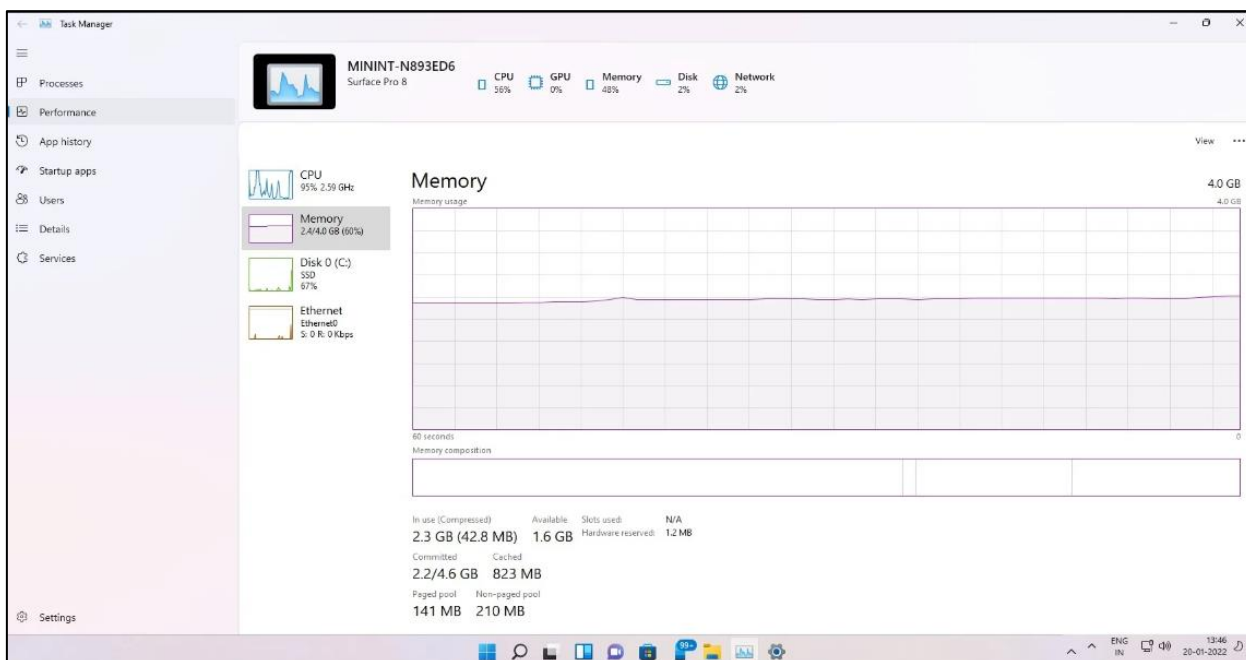


Рисунок 1.13 – Диспетчере задач Windows 11

Однако, как и в случае с любым новым релизом операционной системы, у пользователей могут возникнуть определенные проблемы с совместимостью приложений и адаптацией к новым интерфейсам.

В целом, Windows 11 представляет собой обновление, которое приносит множество новых возможностей и улучшений, и оно, безусловно, продолжит играть важную роль в мире компьютерных технологий в ближайшие годы.

1.3 Анализ функциональности, пользовательского интерфейса и уязвимостей

Диспетчер задач Windows (Task Manager) – это встроенное в операционную систему Windows приложение, предназначенное для управления процессами, мониторинга ресурсов системы и выполнения различных диагностических операций.

Он предоставляет пользователю мощный инструмент для анализа работы операционной системы и её компонентов.

Диспетчер задач Windows обладает широким спектром функций, которые позволяют пользователю контролировать и управлять процессами и ресурсами системы.

Функции диспетчера задач:

1 Одной из основных функций диспетчера задач является возможность просмотра активных процессов и их завершение. Пользователь может видеть список всех запущенных процессов, их идентификаторы, использование CPU и памяти, а также другие характеристики. Также возможно завершение процессов, что позволяет исправлять ситуации, когда приложение зависло или заблокировало систему.

2 Мониторинг ресурсов в диспетчере задач Windows является важной функцией, позволяющей пользователям отслеживать использование различных ресурсов системы и выявлять потенциальные проблемы, связанные с их перегрузкой. Диспетчер задач предоставляет пользователю информацию о текущем использовании центрального процессора (CPU), оперативной памяти (RAM), дискового пространства и сетевых ресурсов. Эта информация представлена в виде графиков и числовых значений, что позволяет легко оценить нагрузку на каждый ресурс.

3 В диспетчере задач есть вкладка "Автозагрузка", которая позволяет пользователю управлять программами, которые запускаются при загрузке системы. Это позволяет оптимизировать время загрузки операционной системы и уменьшить нагрузку на ресурсы.

4 В диспетчере задач есть вкладка "Пользователи", которая позволяет пользователю управлять активными пользователями системы, смотреть информацию, подключать и отключать пользователей.

Дополнительные возможности диспетчера задач Windows значительно расширяют его функциональность и делают его не только инструментом для управления процессами и мониторинга ресурсов, но и универсальным инструментом для анализа работы операционной системы.

Дополнительные возможности диспетчера задач:

1 Диспетчер задач предоставляет пользователю доступ к информации о службах, запущенных на компьютере. Это позволяет узнать, какие службы активны, их статус (запущены, приостановлены, остановлены), а также идентификаторы процессов, связанных с каждой службой. Пользователь может использовать эту информацию для управления службами, например, для запуска, приостановки или остановки определенных служб.

2 Диспетчер задач также предоставляет пользователю информацию о сетевых соединениях, активных на компьютере. Это включает в себя информацию о локальных и удаленных адресах, используемых портах и статусе каждого соединения. Эта функция полезна для мониторинга сетевой активности и выявления потенциальных проблем с соединениями, таких как подозрительные или нежелательные соединения.

3 Диспетчер задач позволяет пользователям создавать и настраивать пользовательские вкладки, чтобы отображать интересующую их информацию. Например, пользователь может настроить вкладку для отображения информации о производительности конкретных приложений или процессов. Это делает диспетчер задач более гибким и адаптивным к потребностям пользователей, позволяя им получать доступ к персонализированной информации о состоянии системы.

Кроме того, диспетчер задач может предоставлять дополнительные функции, такие как отображение информации о подключенных устройствах, общее использование ресурсов системы и прочее.

Эти дополнительные возможности делают диспетчер задач всесторонним инструментом для анализа работы операционной системы и помогают пользователям получать полную картину о её состоянии и производительности.

Диспетчер задач Windows впечатляет своим интуитивно понятным пользовательским интерфейсом, который органично сочетает в себе функциональность и легкость использования.

Благодаря своей элегантной простоте и удобству, этот инструмент обеспечивает непревзойденное взаимодействие с операционной системой, позволяя пользователям контролировать процессы и ресурсы компьютера с легкостью и точностью.

Главное окно диспетчера задач (рисунок 1.14) представляет собой центральное место, где пользователь может наблюдать список текущих активных процессов, а также получать информацию о их характеристиках, таких как использование процессора, памяти и дискового пространства.

Кроме того, здесь предусмотрены управляющие элементы, которые позволяют пользователю осуществлять различные действия, такие как завершение процессов или переключение между вкладками для получения более подробной информации о системе.

Имя	ИД процесса	Имя процесса	8% ЦП	63% Память	1% Диск	2% Сеть	46% GPU	Ядро GPU	Энергопотребление
Приложения (7)									
Проводник	9744	explorer.exe	0,1%	34,4 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое
Диспетчер задач	13528	Taskmgr.exe	3,4%	32,1 МБ	0 МБ/с	0 Мбит/с	0%		Умеренный
Opera Internet Browser (18)			0,1%	717,4 МБ	0,1 МБ/с	0 Мбит/с	0%		Очень низкое
MSYS2 terminal	12292	mintty.exe	0%	0,2 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое
Microsoft Word	22436	WINWORD.EXE	0%	107,6 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое
Microsoft Visual Studio 2022 (7)			0%	422,0 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое
Discord (32 бита) (3)			1,1%	283,3 МБ	0,1 МБ/с	1,9 Мбит/с	45,6%	Графичес...	Низкий
Фоновые процессы (144)									
Экран блокировки Windows п...			0%	0 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое
Хост-процесс для задач Wind...	10940	taskhostw.exe	0%	0,6 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое
Хост-процесс для задач Wind...	7148	taskhostw.exe	0%	1,0 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое
Хост-процесс для задач Wind...	3780	taskhostw.exe	0%	2,9 МБ	0 МБ/с	0 Мбит/с	0%		Очень низкое

Рисунок 1.14 – Главное окно диспетчера задач Windows 10

Диспетчер задач содержит несколько вкладок, каждая из которых отображает определенную информацию о работе системы [11]:

1 Вкладка "Процессы", здесь отображается список всех запущенных процессов на компьютере. Для каждого процесса показывается его имя, PID (идентификатор процесса), использование CPU, памяти и дискового пространства. Пользователь может завершить любой процесс, который он считает ненужным или проблемным.

2 Вкладка "Производительность", на этой вкладке предоставляется дополнительная информация о производительности компьютера. Здесь можно наблюдать графики использования CPU, памяти, диска и сети в реальном времени. Это позволяет пользователям более детально анализировать нагрузку на систему и идентифицировать возможные проблемы производительности.

3 Вкладка "Журнал приложений", здесь пользователь может просмотреть информацию о запущенных приложениях, включая дату и время их запуска, а также любые сообщения или ошибки, связанные с их работой.

Это помогает пользователям отслеживать историю использования приложений и идентифицировать возможные проблемы с ними.

4 Вкладка "Автозагрузка", на этой вкладке отображается список приложений, которые автоматически запускаются при загрузке операционной системы. Пользователи могут просматривать этот список и управлять автозагрузкой приложений, добавляя новые или удаляя ненужные.

5 Вкладка "Пользователи", на этой вкладке отображается список активных пользователей, которые так же могут взаимодействовать с этой операционной системой. Администратор системы может настраивать права подключенных пользователей, подключать новых или отключать старых.

6 Вкладка "Службы" (рисунок 1.15), здесь предоставляется информация о службах операционной системы Windows. Пользователи могут просматривать список всех служб, их статус (запущенные, приостановленные), а также тип их запуска (автоматический, ручной, отключенный). Это позволяет пользователям контролировать работу системных служб и вносить необходимые изменения.

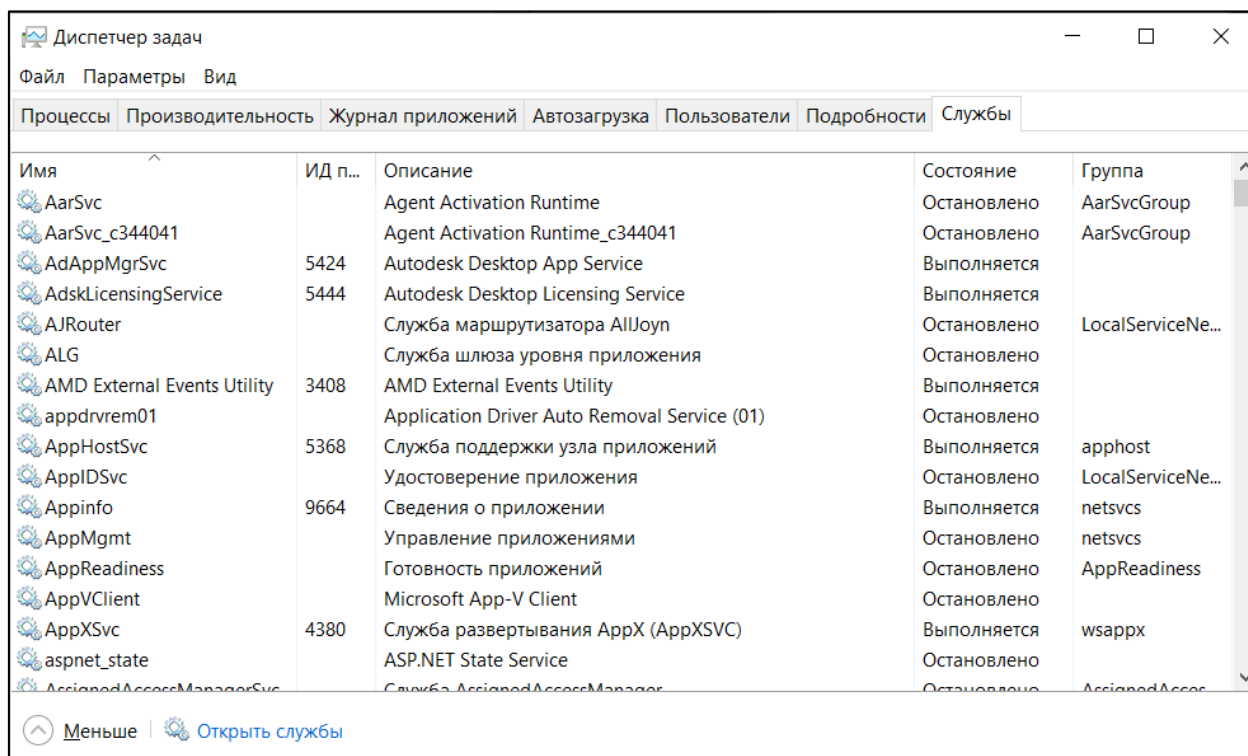


Рисунок 1.15 – Вкладка служб в диспетчере задач Windows 10

Для каждого процесса в списке доступно контекстное меню (рисунок 1.16), которое содержит различные действия, такие как завершение процесса, изменение приоритета и другие. Это обеспечивает удобный и быстрый доступ

к основным операциям.

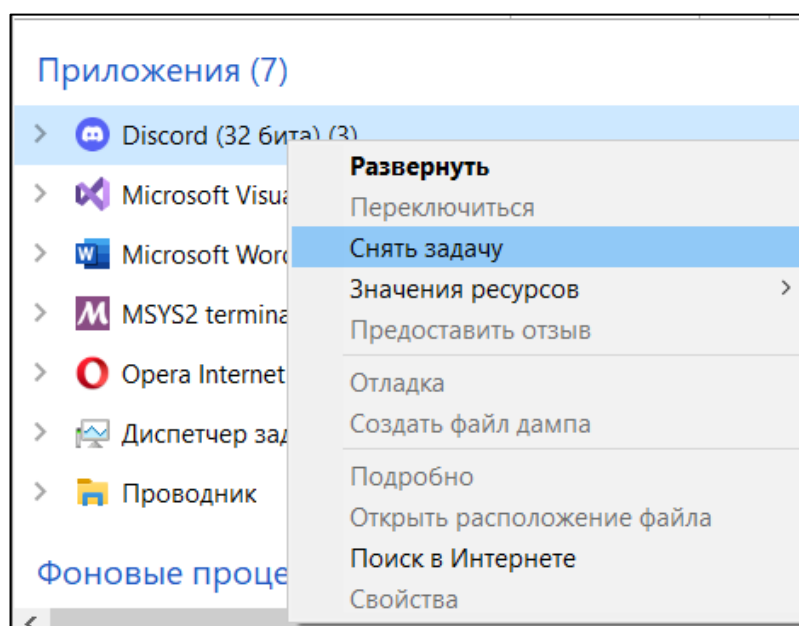


Рисунок 1.16 – Контекстное меню процесса

Диспетчер задач также содержит панели инструментов, которые предоставляют дополнительные возможности управления и мониторинга системы. Например, панель инструментов "Файл" (рисунок 1.17) позволяет запускать новые задачи или закрывать приложения.

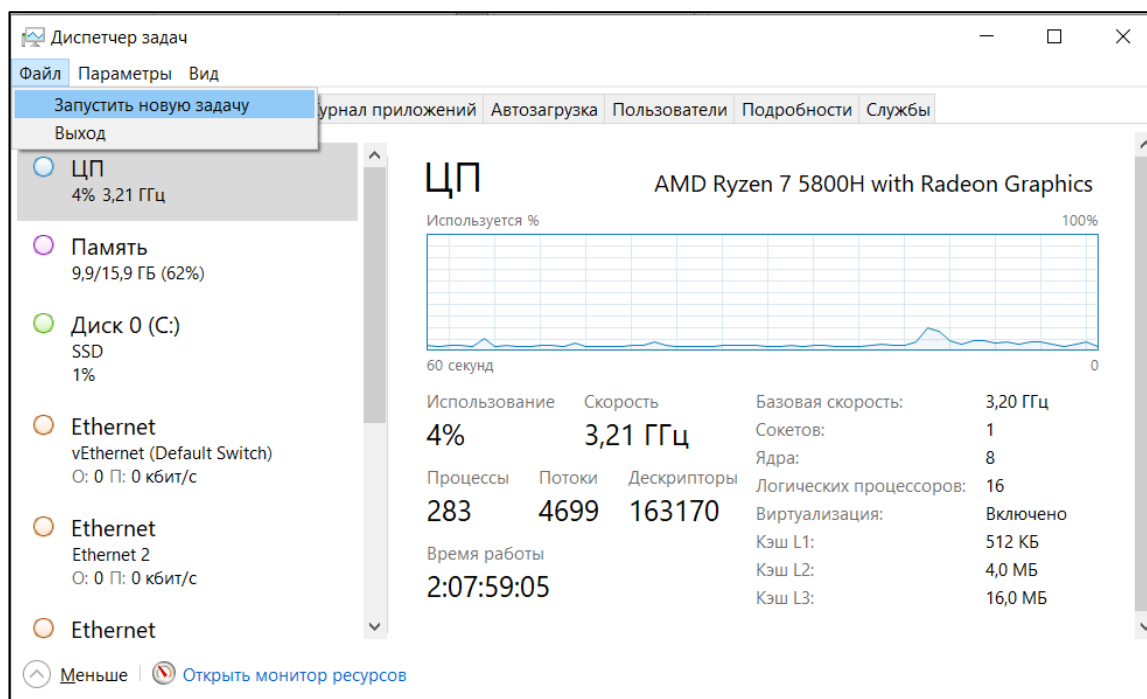


Рисунок 1.17 – Контекстное меню процесса

Несмотря на свою функциональность и удобство использования, диспетчер задач Windows также имеет некоторые уязвимости, которые могут быть использованы злоумышленниками для атаки на систему

Основные виды уязвимостей:

1 Злоумышленники могут использовать диспетчер задач для запуска новых процессов и внедрения в них вредоносного кода. Это может быть особенно опасно, поскольку внедренный код может иметь полный доступ к ресурсам системы и выполнить различные вредоносные действия, такие как сбор и передача конфиденциальной информации или повреждение файлов.

2 Злоумышленники могут изменять имя или другие характеристики вредоносного процесса, чтобы скрыть его под легитимный процесс. Это усложняет обнаружение и удаление вредоносного ПО, поскольку обычные методы мониторинга процессов могут быть обмануты.

3 Злоумышленники могут использовать диспетчер задач для мониторинга активности пользователей и сбора конфиденциальной информации. Например, они могут запустить процесс, который перехватывает нажатия клавиш или снимает скриншоты экрана, чтобы получить доступ к паролям, данным банковских карт или другой чувствительной информации.

Эти уязвимости подчеркивают важность регулярного мониторинга и обновления системы, а также использования дополнительных средств безопасности, чтобы предотвратить возможные атаки.

Диспетчер задач Windows – это мощный инструмент для управления процессами и ресурсами системы. Он обладает широким спектром функций и обеспечивает удобный пользовательский интерфейс.

Однако, как и любое другое приложение, он подвержен различным уязвимостям, которые могут быть использованы злоумышленниками.

Пользователям следует быть осторожными при использовании диспетчера задач и принимать меры для защиты своей системы от возможных атак.

2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Структура и архитектура платформы

Для выполнения данного курсового проекта была выбрана платформа программного обеспечения *Windows 10*. Ниже рассмотрены основные аспекты и характеристики этой платформы, разработанной корпорацией *Microsoft*, а также её роль в разработке и выполнении прикладных программ.

Современная платформа программного обеспечения *Windows* имеет модульную структуру, что позволяет ей эффективно управлять ресурсами многоядерных процессоров и обеспечивать высокую производительность при выполнении параллельных вычислений. ОС *Windows* состоит из следующих основных компонентов [12]:

1 Ядро *Windows (Windows Kernel)* – центральная часть операционной системы, обеспечивающая приложениям координированный доступ к ресурсам компьютера, таким как процессорное время, память, внешнее аппаратное обеспечение, внешнее устройство ввода и устройства вывода. Также обычно ядро предоставляет сервисы файловой системы и сетевых протоколов. Как основополагающий элемент ядро представляет собой наиболее низкий уровень абстракции для доступа приложений к ресурсам системы, необходимым для своей работы. Как правило, ядро предоставляет такой доступ исполняемым процессам соответствующих приложений за счёт использования механизмов межпроцессного взаимодействия и обращения приложений к системным вызовам ОС.

2 Оболочка *Windows (Windows Shell)* – это графический пользовательский интерфейс операционной системы *Windows*, который предоставляет пользователям средства для взаимодействия с компьютером и управления файлами и приложениями. Она является ключевой частью пользовательского опыта в *Windows* и предоставляет удобный способ доступа к функциям ОС.

3 Службы *Windows (Windows Services)* – представляют собой фоновые приложения, которые работают в операционной системе *Windows* независимо от активных пользовательских сессий. Они выполняют разнообразные задачи и функции, которые помогают обеспечивать стабильную и безопасную работу операционной системы. Например, служба обновлений *Windows*, служба антивирусной защиты.

2.2 История, версии и достоинства

Операционная система *Windows* является одной из самых значимых и влиятельных операционных систем в истории компьютерной индустрии. Её история началась в далеком 1985 году с выпуска первой версии, и с тех пор *Windows* прошла долгий путь развития, став одной из самых популярных ОС в мире.

В 1985 году *Microsoft* выпустила первую версию операционной системы *Windows*, *Windows 1.0*. Эта версия представляла собой графическую оболочку для *MS-DOS*, который был распространенной операционной системой на персональных компьютерах в то время. *MS-DOS* (*Microsoft Disk Operating System*) представлял собой командную строку, где пользователи могли вводить команды текстовым образом для выполнения задач.

Однако *Windows 1.0* внесла революцию, добавив элементы графического интерфейса, такие как окна, иконки, меню и диалоговые окна, что сделало использование компьютеров более интуитивным и доступным для широкой аудитории [13].

Несмотря на ограниченные возможности по сравнению с более поздними версиями *Windows*, *Windows 1.0* заложила важные основы для будущего развития графических пользовательских интерфейсов в операционных системах.

Windows 3.0, выпущенная в 1990 году, стала первой версией *Windows*, которая стала самостоятельной операционной системой, а не просто оболочкой для *MS-DOS*. Она включала поддержку *TrueType* шрифтов (формат компьютерных шрифтов, разработанный корпорацией *Apple* в конце 1980-х годов), что улучшило качество текста, а также предоставила пользователю больше возможностей для одновременного выполнения приложений.

Windows 95 (1995 год) была одной из самых значимых версий *Windows*. Она представила новый пользовательский интерфейс с кнопкой "Пуск" и более удобное управление файлами. Одной из ключевых особенностей *Windows 95* была поддержка *Plug and Play (PnP)*. Эта технология позволяла компьютеру автоматически обнаруживать новое аппаратное обеспечение, устанавливать необходимые драйверы и настраивать устройства. Это существенно упростило процесс добавления нового оборудования и сделало его более доступным для пользователей.

Windows XP, выпущенная в 2001 году, стала одной из самых популярных версий *Windows* в истории. Она предоставила стабильность и удобство использования, а также улучшила поддержку сетей, что сделало её идеальным

выбором как для домашних, так и для корпоративных пользователей.

Windows 7, выпущенная в 2009 году, заслужила признание благодаря своей высокой стабильности, улучшенной производительности и интуитивно понятному интерфейсу. Эта операционная система была одной из наиболее популярных версий *Windows* и нашла широкое применение как среди домашних пользователей, так и среди бизнес-клиентов. Улучшенная поддержка аппаратного обеспечения, удобная организация рабочего стола и стабильная работа сделали *Windows 7* предпочтительным выбором для многих пользователей, и она оставалась востребованной долгое время до выпуска последующих версий операционной системы *Windows*.

Windows 10 внедрила "сервисную" модель обновлений, которая предполагает регулярные обновления операционной системы. Это означает, что *Microsoft* регулярно выпускает крупные обновления, обогащая функциональность и улучшая безопасность ОС. Вместо того чтобы выпускать новые версии *Windows* каждые несколько лет, компания предоставляет обновления с новыми функциями и исправлениями прямо через интернет. Это позволяет пользователям оставаться на актуальной версии ОС и получать новые возможности [14].

Windows 11 представила совершенно новый пользовательский интерфейс с улучшенными анимациями, более современным дизайном и улучшенной оптимизацией для сенсорных устройств. Этот интерфейс создает более современный и стильный вид операционной системы. Одной из значимых нововведений *Windows 11* является поддержка приложений для *Android*. Пользователи могут устанавливать и запускать приложения из магазина приложений *Android* на своих устройствах с *Windows 11*, что расширяет функциональность и доступность приложений.

Обе версии, *Windows 10* и *Windows 11*, продолжают совершенствоваться и развивать пользовательский опыт, предоставляя новые возможности и усовершенствования. Эти операционные системы оставляют прочный след в истории *Windows* и удовлетворяют потребности современных пользователей.

Весь этот путь развития *Windows* свидетельствует о постоянной эволюции операционной системы и стремлении *Microsoft* к улучшению производительности, безопасности и удобства использования для миллионов пользователей по всему миру. *Windows* остается одной из наиболее популярных и важных операционных систем на сегодняшний день.

2.3 Обоснование выбора платформы

В ходе курсового проекта выбор программной платформы играет ключевую роль. В данном случае, было принято решение использовать операционную систему *Windows 10* в качестве платформы для разработки *Windows Task Manager* [15].

Существует несколько важных аргументов, которые обосновывают выбор *Windows 10*:

1 Широкое распространение. Операционная система *Windows* является одной из наиболее распространенных операционных систем в мире. Это означает, что множество пользователей и организаций работают на этой платформе. Такой широкий пользовательский базис обеспечивает актуальность и практическую значимость исследования.

2 Поддержка многих процессоров. *Windows* обладает высокой совместимостью с различными аппаратными конфигурациями, включая многоядерные процессоры. Это предоставляет отличную возможность для исследования и оптимизации вычислений на многопроцессорных системах.

3 Богатая среда разработки. *Windows* предоставляет разработчикам обширный набор инструментов для разработки прикладных программ. Включая *Visual Studio* и поддержку для различных языков программирования. Эти инструменты значительно упрощают процесс создания и тестирования параллельных приложений.

Таким образом, выбор операционной системы *Windows 10* в качестве платформы для разработки аналога *Windows Task Manager* обоснован на основе ее широкой распространенности, совместимости с многоядерными процессорами, наличии разнообразных инструментов разработки, богатстве библиотек и простоте использования.

Эта платформа обеспечивает удовлетворение требований исследования и позволяет провести обширное исследование в области разработки приложения, способного анализировать список процессов системы *Windows*, их характеристик и управления.

3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА

3.1 Обоснование необходимости разработки

Разработка диспетчера задач Windows – это не только техническое исполнение, но и фундаментальная составляющая эволюции операционных систем. Её необходимость вытекает из постоянного стремления к оптимизации и повышению эффективности работы компьютерных систем, что в свою очередь обеспечивает более гладкую и удобную работу пользователя.

Управление процессами и ресурсами – это основополагающий аспект функционирования операционных систем.

Диспетчер задач играет важную роль в обеспечении контроля за процессами, обрабатываемыми системой, и эффективным распределением ресурсов.

Без должного управления этими процессами, система может столкнуться с проблемами, такими как перегрузка CPU, избыточное потребление памяти и другие факторы, приводящие к снижению производительности и ухудшению опыта пользователя.

Мониторинг ресурсов – это необходимый инструмент для выявления проблем с производительностью и обеспечения эффективного использования ресурсов.

Системный мониторинг позволяет оперативно реагировать на изменения в использовании CPU, памяти, дискового пространства и сетевых ресурсов.

Благодаря диспетчеру задач пользователь может наблюдать за состоянием ресурсов и принимать меры для их оптимизации, предотвращая возможные проблемы.

Диагностика и анализ – это важная часть обеспечения стабильности и производительности операционной системы.

Диспетчер задач предоставляет пользователям инструменты для выявления и анализа проблемных процессов, а также определения причин снижения производительности. Это позволяет пользователям принимать меры для устранения проблем и повышения эффективности работы системы.

Универсальность и удобство использования – это ключевые принципы, которыми руководствуются разработчики при создании диспетчера задач Windows [16]. Интуитивно понятный интерфейс делает его доступным для широкого круга пользователей, что позволяет им эффективно управлять и

мониторить работоспособность операционной системы.

Благодаря постоянному совершенствованию этого инструмента пользователи получают удобные и эффективные средства для управления своими компьютерными системами.

С развитием технологий и появлением новых возможностей операционных систем, необходимость в совершенствовании диспетчера задач становится все более актуальной. Разработчики должны постоянно адаптировать инструмент под современные требования и предоставлять пользователям новые функции и возможности для эффективного управления процессами и ресурсами.

В современном мире безопасность компьютерных систем является одним из главных приоритетов. Разработка диспетчера задач Windows также обусловлена необходимостью обеспечения защиты от вредоносных программ и атак злоумышленников.

Постоянное совершенствование и улучшение механизмов безопасности помогает предотвращать угрозы и обеспечивать безопасность данных и конфиденциальную информацию пользователей [17].

В общем, разработка диспетчера задач Windows обусловлена не только стремлением к повышению функциональности операционной системы, но и стремлением к обеспечению удобства использования и эффективности работы пользователя. Этот инструмент играет ключевую роль в обеспечении стабильной и эффективной работы компьютерной системы, обеспечивая при этом комфортное и продуктивное взаимодействие с пользователем.

3.2 Технологии программирования, используемые для решения поставленных задач

Разработка аналога диспетчера задач для операционной системы Windows – это глубокий и многосторонний процесс, который требует внимательного изучения и использования различных технологий программирования [18].

Язык программирования C# широко используется для создания различных приложений, включая аналоги диспетчера задач.

Основные причины, почему C# является предпочтительным выбором для разработки аналога диспетчера задач:

1 C# предоставляет разработчикам широкий спектр инструментов и возможностей для решения различных задач. Он поддерживает объектно-ориентированное программирование (ООП), обобщённые типы (generics), делегаты и события, асинхронное программирование и многое другое, что делает его гибким и мощным инструментом для создания сложных приложений.

2 C# разработан для работы на платформе .NET, которая предоставляет богатую стандартную библиотеку классов (BCL) и множество других инструментов для разработки приложений под Windows. Это обеспечивает удобство в работе с файловой системой, сетевыми запросами, обработкой данных и многими другими задачами.

3 В современных приложениях диспетчера задач часто требуется обрабатывать множество операций одновременно. C# предоставляет удобные средства для асинхронного программирования, такие как ключевые слова `async/await`, которые позволяют эффективно управлять асинхронными операциями без блокирования основного потока выполнения.

4 Благодаря платформе .NET, приложения на C# можно запускать не только на Windows, но и на других операционных системах, таких как Linux и macOS. Это расширяет аудиторию пользователей и обеспечивает большую гибкость в развертывании приложений.

WinForms (Windows Forms) – это технология разработки пользовательских интерфейсов (UI) для приложений под платформу Windows.

Основные причины, по которым WinForms является популярным выбором для создания приложений с графическим интерфейсом на C#:

1 WinForms предоставляет богатый набор стандартных элементов управления, таких как кнопки, текстовые поля, метки, списки, таблицы и многое другое. Это позволяет разработчикам легко создавать интерфейсы с

использованием знакомых и привычных элементов.

2 Одним из главных преимуществ WinForms является его простота использования. Разработчики могут быстро создавать и настраивать пользовательские интерфейсы с помощью визуального редактора в среде разработки Visual Studio. Это делает процесс разработки более эффективным и удобным.

3 Благодаря простоте и интуитивно понятному интерфейсу WinForms, разработчики могут быстро создавать функциональные приложения без необходимости писать большое количество кода вручную. Это особенно полезно для небольших и средних проектов, где важно минимизировать время разработки.

Однако для работы с системными ресурсами и процессами необходимо использовать специализированные технологии.

WMI (Windows Management Instrumentation) является мощным инструментом для взаимодействия с различными компонентами операционной системы Windows [19].

Основные причины использования WMI:

1 WMI предоставляет разработчикам доступ к различным системным ресурсам, таким как процессы, службы, жёсткие диски, сетевые подключения, операционная система и многое другое. Это позволяет получать информацию о текущем состоянии системы, её конфигурации и использовании ресурсов.

2 С помощью WMI разработчики могут мониторить запущенные процессы, получать информацию о них (например, идентификатор процесса, потребление памяти, время CPU и т. д.) и выполнять различные операции управления, такие как запуск, приостановка или завершение процессов.

3 WMI позволяет управлять службами операционной системы Windows, включая их запуск, остановку, приостановку, возобновление и изменение параметров конфигурации. Это полезно для автоматизации управления службами и обеспечения их надлежащей работы.

WMI предоставляет удобный интерфейс для взаимодействия с операционной системой Windows из приложений, написанных на языке C#. Существуют специальные библиотеки и классы .NET Framework, такие как ManagementObject и ManagementScope, которые упрощают работу с WMI из кода на C# [20].

Кроме того, для доступа к дополнительным функциям и ресурсам операционной системы могут использоваться различные библиотеки и DLL (Dynamic Link Libraries):

1 kernel32.dll – библиотека предоставляет доступ к множеству функций,

связанных с управлением операционной системой на низком уровне. Она содержит функции для управления памятью (например, выделение и освобождение памяти), работой с процессами (например, создание, открытие и завершение процессов) и потоками (например, создание и управление потоками выполнения).

2 user32.dll – библиотека содержит функции, отвечающие за создание и управление графическим интерфейсом пользователя. Она предоставляет доступ к функциям для создания окон, обработки сообщений окон, управления ресурсами окон (например, рисование, изменение размеров и позиции окон) и другим аспектам пользовательского интерфейса.

3 advapi32.dll – библиотека содержит функции, связанные с безопасностью, управлением учетными записями пользователей, работой с реестром и другими аспектами системной администрации. Она предоставляет доступ к функциям для работы с службами, аутентификации пользователей, шифрования данных и другими средствами защиты информации.

4 shell32.dll - библиотека содержит функции, связанные с работой оболочки операционной системы Windows. Она предоставляет доступ к функциям для работы с файловой системой (например, создание, копирование, перемещение и удаление файлов), управлением ярлыками, работой с контекстным меню и другими функциями, связанными с оболочкой Windows.

Таким образом, выбранные технологии программирования - C#, WinForms, WMI, WinApi и Windows DLL - обеспечивают широкие возможности для разработки функционального и эффективного аналога диспетчера задач для операционной системы Windows. Их совместное использование позволяет создать приложение, которое будет обладать удобным пользовательским интерфейсом и обеспечивать полный контроль над процессами и ресурсами системы.

4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГРАММЫ

4.1 Разработка API

Для разработки аналога Windows Task Manager, ключевым аспектом является создание API, которое обеспечивает взаимодействие с различными системными компонентами и функциями операционной системы.

Важно иметь надежный и эффективный способ получения информации о процессах, службах, ресурсах и других системных параметрах, а также возможность управления ими.

Для этих целей, был разработан частичный класс API (рисунок 4.1, рисунок 4.2) который обеспечивают доступ к необходимым системным ресурсам и операциям.

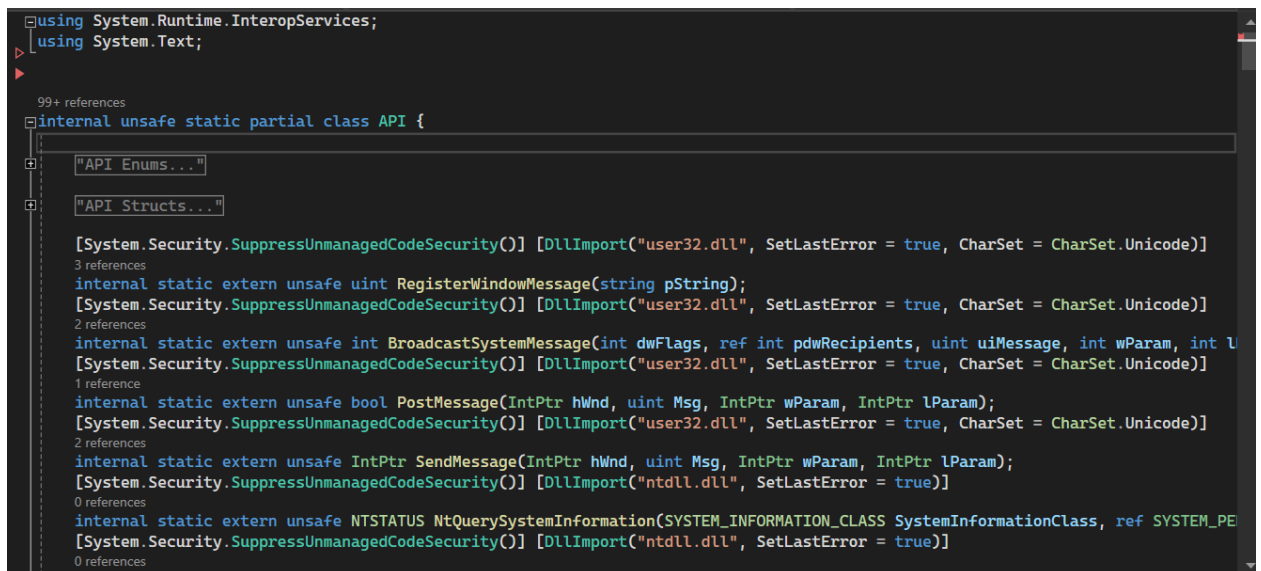


Рисунок 4.1 – Первая часть класса API

В представленном API, использованном в диспетчере задач, важную роль играют функции, импортированные из различных системных библиотек, таких как user32.dll, kernel32.dll, advapi32.dll и других. Эти функции предоставляют доступ к различным системным ресурсам и операциям, таким как управление процессами, получение информации о производительности, работа с службами и т.д.

Одним из ключевых элементов API является возможность получения информации о процессах. Для этого используются функции, такие как EnumProcesses, OpenProcess, GetProcessMemoryInfo и другие.

Эти функции позволяют получить информацию о процессах, включая их идентификаторы, использование памяти, приоритет и другие характеристики.



Рисунок 4.2 – Вторая часть класса API

Для работы с процессами также необходимы функции управления ими, такие как `TerminateProcess`, `SuspendThread`, `ResumeThread` и другие. Они позволяют завершать процессы, приостанавливать и возобновлять выполнение потоков процессов, управлять приоритетом и аффинитетом процессов и т.д.

Для получения информации о системе и её производительности используются функции, такие как `GetSystemInfo`, `GlobalMemoryStatusEx`, `GetSystemTimes` и другие. Эти функции предоставляют информацию о системных ресурсах, загрузке процессора, использовании памяти и т.д.

Взаимодействие с системными службами также важно для функционирования аналога диспетчера задач. Для этого используются функции, такие как `OpenSCManager`, `OpenService`, `StartService`, `ControlService` и другие. Они позволяют получать информацию о службах, их статусе, запускать и останавливать службы, управлять ими и т.д.

Помимо этого, API предоставляет функциональность для работы с окнами и процессами пользовательского интерфейса.

Для этого используются функции, такие как `EnumWindows`, `GetWindowInfo`, `GetWindowText` и другие. Они позволяют получать информацию о открытых окнах, их заголовках, состоянии и т.д.

В целом, разработка аналога Windows Task Manager требует создания комплексного API, которое обеспечивает доступ к различным системным ресурсам и операциям, а также возможность управления ими.

При проектировании такого API необходимо учитывать требования к производительности, надежности и безопасности, а также обеспечивать

удобный интерфейс для взаимодействия с ним.

Так же для эффективной разработки аналога диспетчера задач Windows, был разработан класс Handles (рисунок 4.3), который играет важную роль в разработке API.

```
1 using System.Text;
2 using System.Diagnostics;
3 using System.Runtime.InteropServices;
4 using System.Runtime.Versioning;
5
6
7 [SupportedOSPlatform("windows")]
8 internal unsafe static partial class Handles {
9     const int MAX_PATH = 260;
10    const int DUPLICATE_SAME_ACCESS = 0x2;
11    const int CNST_SYSTEM_HANDLE_INFORMATION = 16;
12    const uint STATUS_INFO_LENGTH_MISMATCH = 0xC0000004U;
13
14    0 references
15    public static List<string> GetFilesLockedBy(Process process) {
16        try {
17            var retFiles = new List<string>();
18            foreach (SYSTEM_HANDLE_INFORMATION h in GetHandles(process)) {
19                try {
20                    var File = GetFilePath(h, process);
21                    if (File != null) {
22                        retFiles.Add(File);
23                    }
24                } catch (Exception ex) { Shared.DebugTrap(ex, 1901); }
25            }
26            return retFiles;
27        } catch (Exception ex) { Shared.DebugTrap(ex, 1902); return new List<string>(); }
28    }
29 }
```

Рисунок 4.3 – Класс Handles

Он предоставляет функции для работы с дескрипторами объектов в системе, что необходимо для получения информации о процессах и файловых ресурсах, заблокированных этими процессами.

Задачи, которые решает этот класс:

1 Получение списка файлов, заблокированных процессом: Метод GetFilesLockedBy принимает процесс и возвращает список файлов, к которым этот процесс имеет открытый дескриптор.

2 Получение списка дескрипторов объектов, используемых процессом: Метод GetHandles позволяет получить список дескрипторов объектов, которые использует указанный процесс.

3 Получение пути к файлу по дескриптору объекта: Метод GetFilePath получает путь к файлу, соответствующему указанному дескриптору объекта, используя системные вызовы для работы с объектами операционной системы.

4 Преобразование внутреннего ключа реестра в обычный формат: Метод GetRegularKeyName преобразует внутренний ключ реестра в формат, понятный для пользователя.

Кроме того, класс содержит структуры и перечисления, необходимые для работы с системными функциями и типами данных, а также использует

внешние библиотеки через платформо-зависимые вызовы, такие как `DllImport` (рисунок 4.4).

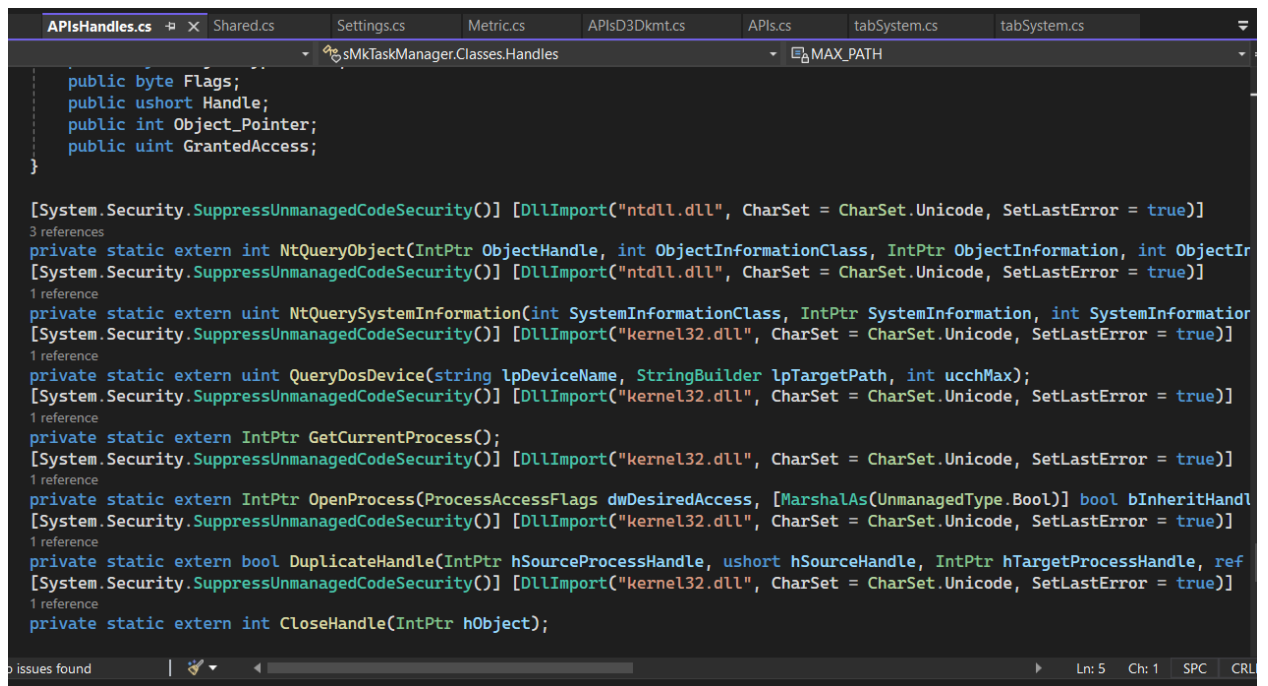


Рисунок 4.4 – Примеры объявлений вызовов `DllImport`

Таким образом, класс `Handles` является важным компонентом API для взаимодействия с системными ресурсами и объектами операционной системы, что позволяет аналогу Windows Task Manager получать информацию о процессах и управлять ими.

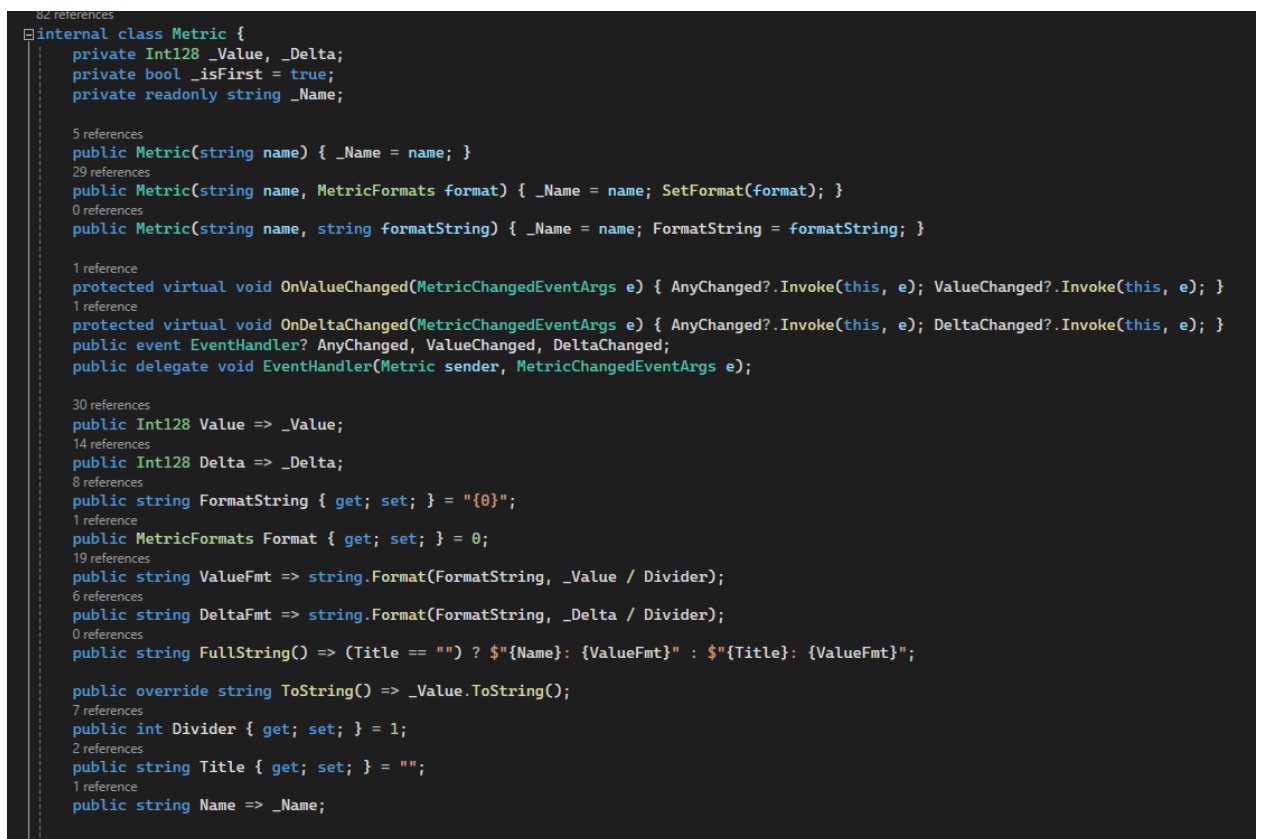
4.2 Вспомогательные классы

В данной главе представлены вспомогательные классы, используемые для разработки аналога Windows Task Manager.

Класс `Metric` (рисунок 4.5) представляет метрику с определенным форматом отображения.

Он содержит следующие элементы:

- 1 `Value` представляет текущее значение метрики
- 2 `Delta` - строка формата для отображения значения метрики
- 3 `Format` определяет формат метрики (`None`, `Numeric`, `Kb`, `Mb`, `Gb`)
- 4 `Divider` служит для преобразования значения метрики
- 5 `Title` - это заголовок метрики
- 6 `Name` – имя метрики



```
internal class Metric {
    private Int128 _Value, _Delta;
    private bool _isFirst = true;
    private readonly string _Name;

    5 references
    public Metric(string name) { _Name = name; }
    29 references
    public Metric(string name, MetricFormats format) { _Name = name; SetFormat(format); }
    0 references
    public Metric(string name, string formatString) { _Name = name; FormatString = formatString; }

    1 reference
    protected virtual void OnValueChanged(MetricChangedEventArgs e) { AnyChanged?.Invoke(this, e); ValueChanged?.Invoke(this, e); }
    1 reference
    protected virtual void OnDeltaChanged(MetricChangedEventArgs e) { AnyChanged?.Invoke(this, e); DeltaChanged?.Invoke(this, e); }
    public event EventHandler? AnyChanged, ValueChanged, DeltaChanged;
    public delegate void EventHandler(Metric sender, MetricChangedEventArgs e);

    30 references
    public Int128 Value => _Value;
    14 references
    public Int128 Delta => _Delta;
    8 references
    public string FormatString { get; set; } = "{0}";
    1 reference
    public MetricFormats Format { get; set; } = 0;
    19 references
    public string ValueFmt => string.Format(FormatString, _Value / Divider);
    6 references
    public string DeltaFmt => string.Format(FormatString, _Delta / Divider);
    0 references
    public string FullString() => (Title == "") ? $"{Name}: {ValueFmt}" : $"{Title}: {ValueFmt}";

    public override string ToString() => _Value.ToString();
    7 references
    public int Divider { get; set; } = 1;
    2 references
    public string Title { get; set; } = "";
    1 reference
    public string Name => _Name;
}
```

Рисунок 4.5 – Класс `Metric`

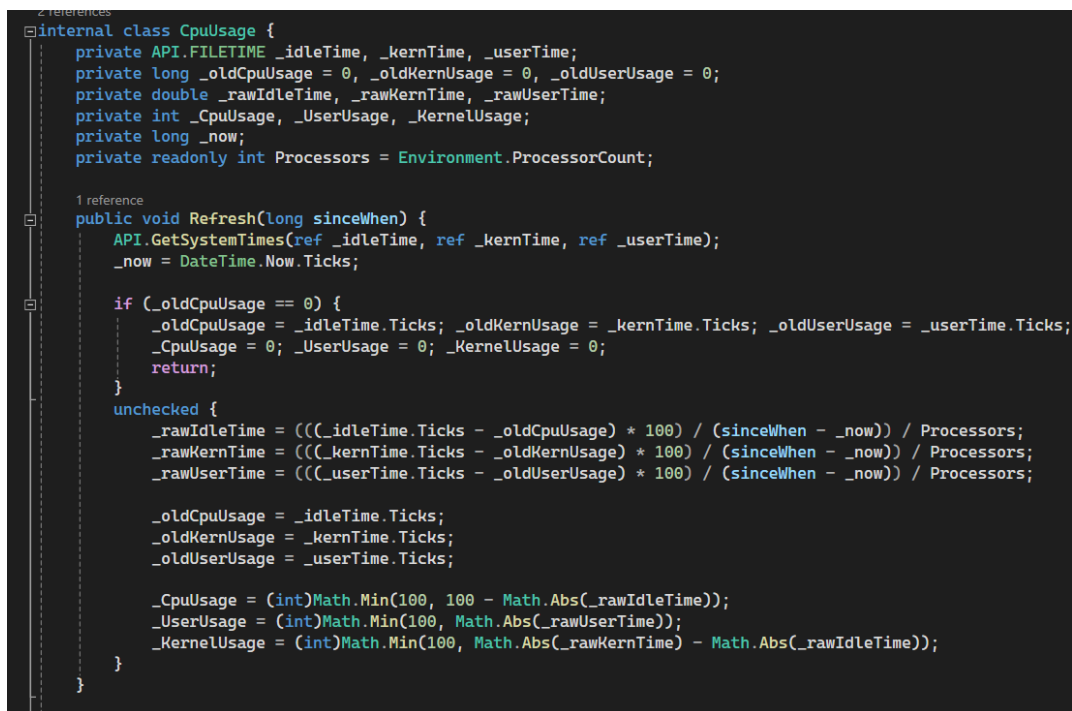
В классе `Metric` также реализованы события `AnyChanged`, `ValueChanged` и `DeltaChanged`, а также соответствующие методы для их обработки. Также предоставлены методы для установки формата метрики, установки нового значения и инкрементирования значения метрики.

Класс `MetricNew` является обобщенной версией класса `Metric`, который поддерживает работу с различными типами данных. Он также предоставляет функционал для установки формата метрики, установки нового значения и инкрементирования значения метрики.

Классы `MetricChangedEventArgs` и `MetricNewChangedEventArgs` используются для передачи информации о изменении метрик.

Классы `HighlightsClass`, `PerformanceClass`, `NetworkingClass` и `ProcessDetailsClass` представляют настройки для различных аспектов приложения, таких как подсветка, производительность, сеть и детали процессов.

Класс `CpuUsage` представляет функционал для отслеживания использования центрального процессора (рисунок 4.6). Он предоставляет информацию о загрузке процессора, пользовательском и ядре.



```
internal class CpuUsage {
    private API.FILETIME _idleTime, _kernTime, _userTime;
    private long _oldCpuUsage = 0, _oldKernUsage = 0, _oldUserUsage = 0;
    private double _rawIdleTime, _rawKernTime, _rawUserTime;
    private int _CpuUsage, _UserUsage, _KernelUsage;
    private long _now;
    private readonly int Processors = Environment.ProcessorCount;

    1 reference
    public void Refresh(long sinceWhen) {
        API.GetSystemTimes(ref _idleTime, ref _kernTime, ref _userTime);
        _now = DateTime.Now.Ticks;

        if (_oldCpuUsage == 0) {
            _oldCpuUsage = _idleTime.Ticks; _oldKernUsage = _kernTime.Ticks; _oldUserUsage = _userTime.Ticks;
            _CpuUsage = 0; _UserUsage = 0; _KernelUsage = 0;
            return;
        }
        unchecked {
            _rawIdleTime = (((_idleTime.Ticks - _oldCpuUsage) * 100) / (sinceWhen - _now)) / Processors;
            _rawKernTime = (((_kernTime.Ticks - _oldKernUsage) * 100) / (sinceWhen - _now)) / Processors;
            _rawUserTime = (((_userTime.Ticks - _oldUserUsage) * 100) / (sinceWhen - _now)) / Processors;

            _oldCpuUsage = _idleTime.Ticks;
            _oldKernUsage = _kernTime.Ticks;
            _oldUserUsage = _userTime.Ticks;

            _CpuUsage = (int)Math.Min(100, 100 - Math.Abs(_rawIdleTime));
            _UserUsage = (int)Math.Min(100, Math.Abs(_rawUserTime));
            _KernelUsage = (int)Math.Min(100, Math.Abs(_rawKernTime) - Math.Abs(_rawIdleTime));
        }
    }
}
```

Рисунок 4.6 – Класс `CpuUsage`

Класс `Settings` содержит статические поля и методы для работы с настройками приложения, как из конфигурационного файла, так и из реестра.

4.3 Task Manager

Класс **TaskManagerValuesBase** (рисунок 4.7) играет ключевую роль в архитектуре Task Manager, предоставляя базовую функциональность для отслеживания и обработки метрик.

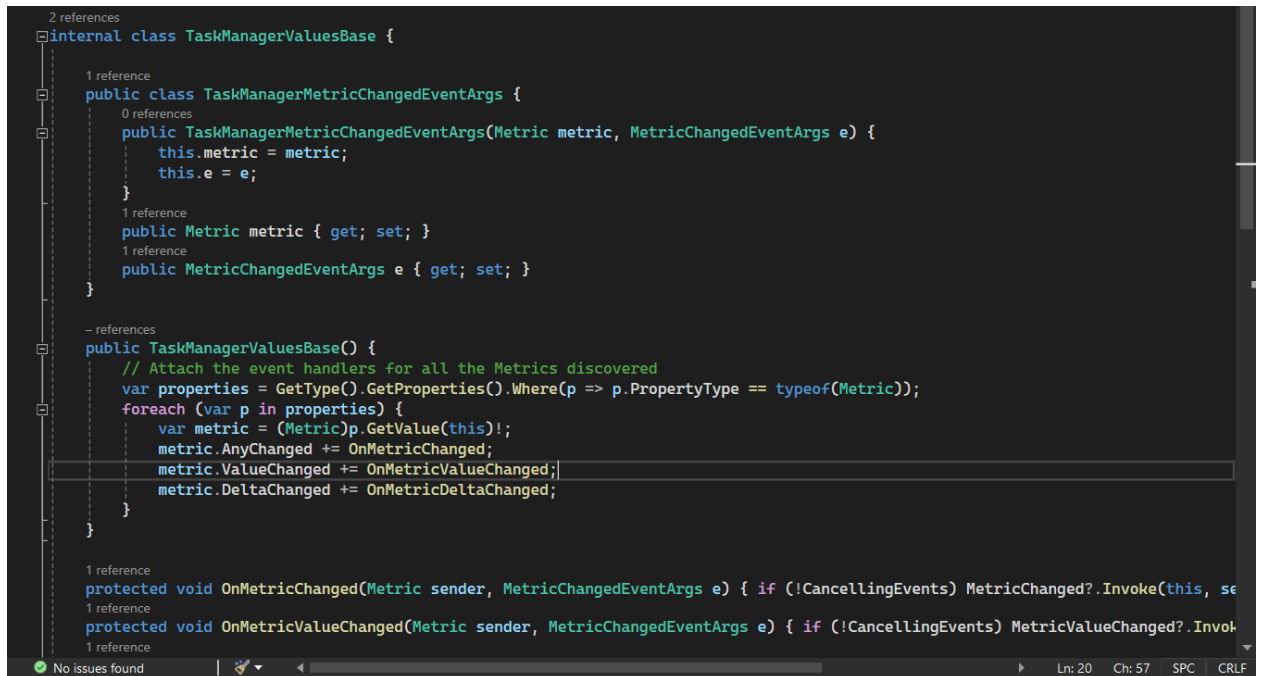


Рисунок 4.7 – Класс `TaskManagerValuesBase`

Внутренний класс **TaskManagerMetricChangedEventArgs** служит для передачи информации о изменении метрики.

Конструктор класса `TaskManagerValuesBase` выполняет несколько важных действий при создании экземпляра класса, он присоединяет обработчики событий для всех метрик, обнаруженных в классе. Это происходит с помощью рефлексии, для каждого свойства типа `Metric` вызывается `GetValue`, и для полученной метрики добавляются обработчики событий `AnyChanged`, `ValueChanged` и `DeltaChanged`.

Методы `OnMetricChanged`, `OnMetricValueChanged` и `OnMetricDeltaChanged` вызывают соответствующие события `MetricChanged`, `MetricValueChanged` и `MetricDeltaChanged` при изменении метрик. При этом проверяется флаг `CancellingEvents`, который может быть использован для временного отключения генерации событий.

События `MetricChanged`, `MetricValueChanged` и `MetricDeltaChanged` объявляют делегаты `MetricEventHandler` и предоставляют возможность подписки на изменения метрик в других классах.

Метод `SetValue` позволяет установить новое значение для заданной метрики.

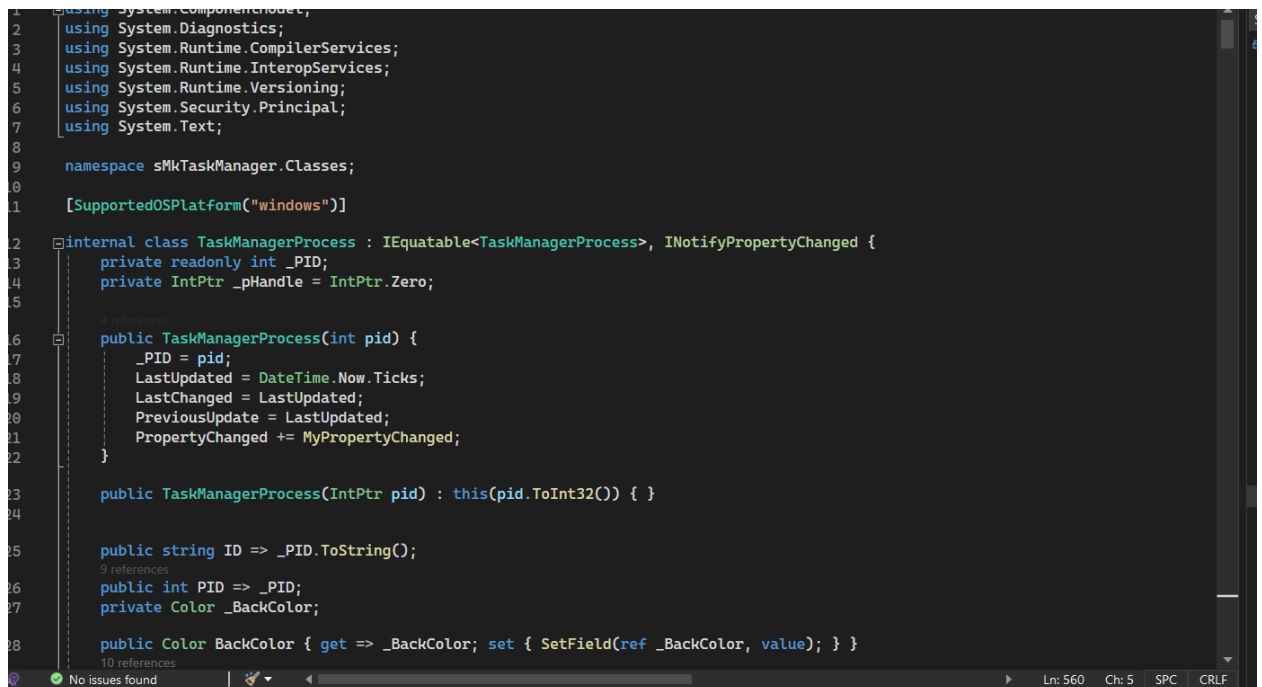
Публичное поле `CancellingEvents` указывает, отключены ли в данный момент генерация событий.

Публичное поле `LastUpdate` хранит информацию о времени последнего обновления метрик.

Класс `TaskManagerValuesBase` предоставляет удобный и гибкий способ управления метриками в `Task Manager`, позволяя обрабатывать их изменения и реагировать на них в соответствии с бизнес-логикой приложения.

В глубинах кода `Task Manager` скрыты не только простые строки инструкций, но и элегантные механизмы, обеспечивающие точное и детальное отображение информации о запущенных процессах. Давайте погрузимся в мир этих классов и рассмотрим их функциональность с точки зрения архитектурного мастерства и непревзойденной эффективности.

В центре внимания стоит класс **`TaskManagerProcess`**, который является ключом к миру процессов операционной системы (рисунок 4.8). Его структура и функциональность созданы для того, чтобы взаимодействовать с информацией о процессах, предоставляемой операционной системой, и отображать ее в удобной форме.



```
1 using System.ComponentModel;
2 using System.Diagnostics;
3 using System.Runtime.CompilerServices;
4 using System.Runtime.InteropServices;
5 using System.Runtime.Versioning;
6 using System.Security.Principal;
7 using System.Text;
8
9 namespace SMkTaskManager.Classes;
10
11 [SupportedOSPlatform("windows")]
12 internal class TaskManagerProcess : IEquatable<TaskManagerProcess>, INotifyPropertyChanged {
13     private readonly int _PID;
14     private IntPtr _pHandle = IntPtr.Zero;
15
16     public TaskManagerProcess(int pid) {
17         _PID = pid;
18         LastUpdated = DateTime.Now.Ticks;
19         LastChanged = LastUpdated;
20         PreviousUpdate = LastUpdated;
21         PropertyChanged += MyPropertyChanged;
22     }
23
24     public TaskManagerProcess(IntPtr pid) : this(pid.ToInt32()) { }
25
26     public string ID => _PID.ToString();
27     public int PID => _PID;
28     private Color _BackColor;
29     public Color BackColor { get => _BackColor; set { SetField(ref _BackColor, value); } }
30 }
```

Рисунок 4.8 – Класс `TaskManagerProcess`

Каждое поле и свойство этого класса несет в себе уникальный смысл и предоставляет ценную информацию о процессе. От идентификатора процесса

(PID) до использования центрального процессора (ЦП) и оперативной памяти - каждый аспект тщательно разработан и предназначен для точного отображения состояния процесса.

TaskManagerProcessCollection представляет собой не просто коллекцию объектов **TaskManagerProcess**, а целостную систему управления процессами (рисунок 4.9). Его методы обеспечивают эффективный поиск и доступ к процессам, что делает его неотъемлемой частью функциональности Task Manager.

```
[SupportedOSPlatform("windows")]
3 references
internal class TaskManagerProcessCollection : BindingList<TaskManagerProcess> {
    2 references
    public BindingSource DataExporter { get; init; }
    1 reference
    public TaskManagerProcessCollection() {
        DataExporter = new(this, null) { RaiseListChangedEvents = true };
        AllowRemove = true;
        RaiseListChangedEvents = true;
    }

    1 reference
    public bool Contains(IntPtr PID) => Contains(PID.ToInt32());
    1 reference
    public bool Contains(int PID) {
        foreach (TaskManagerProcess i in Items) {
            if (i.PID == PID) return true;
        }
        return false;
    }

    0 references
    public bool Contains(IntPtr PID, ref TaskManagerProcess theProcess) => Contains(PID.ToInt32(), ref theProcess);
    1 reference
    public bool Contains(int PID, ref TaskManagerProcess theProcess) {
        foreach (TaskManagerProcess i in Items) {
            if (i.PID == PID) { theProcess = i; return true; }
        }
        return false;
    }
}
```

Рисунок 4.8 – Класс TaskManagerProcessCollection

TaskManagerProcessCollection предоставляет удобные методы для поиска процессов по их идентификатору и получения объектов TaskManagerProcess. Это упрощает процесс управления процессами и обеспечивает быстрый доступ к нужным данным.

Этот класс интегрируется с пользовательским интерфейсом Task Manager, обеспечивая плавное и эффективное взаимодействие между представлением данных и их обработкой. Он служит связующим звеном между внутренними механизмами системы и пользовательским опытом.

TaskManagerProcess и **TaskManagerProcessCollection** – это не просто классы в коде, это ключи к пониманию работы операционной системы на уровне процессов. Их структура, методы и события обеспечивают надежное и эффективное взаимодействие с информацией о процессах, делая Task Manager мощным инструментом анализа и контроля производительности системы.

5 ВЗАИМОДЕЙСТВИЕ С ПРОГРАММНЫМ ПРОДУКТОМ

5.1 Общие возможности программы

Task Manager сочетает в себе функциональность мониторинга, анализа и управления, позволяя пользователям эффективно контролировать работу своих процессов.

Когда пользователь запускает Task Manager, программа открывает главное окно, которое является центром управления процессами и ресурсами компьютера. Это окно представляет собой наглядный и информативный интерфейс, отображающий основные характеристики персонального компьютера (рисунок 5.1).

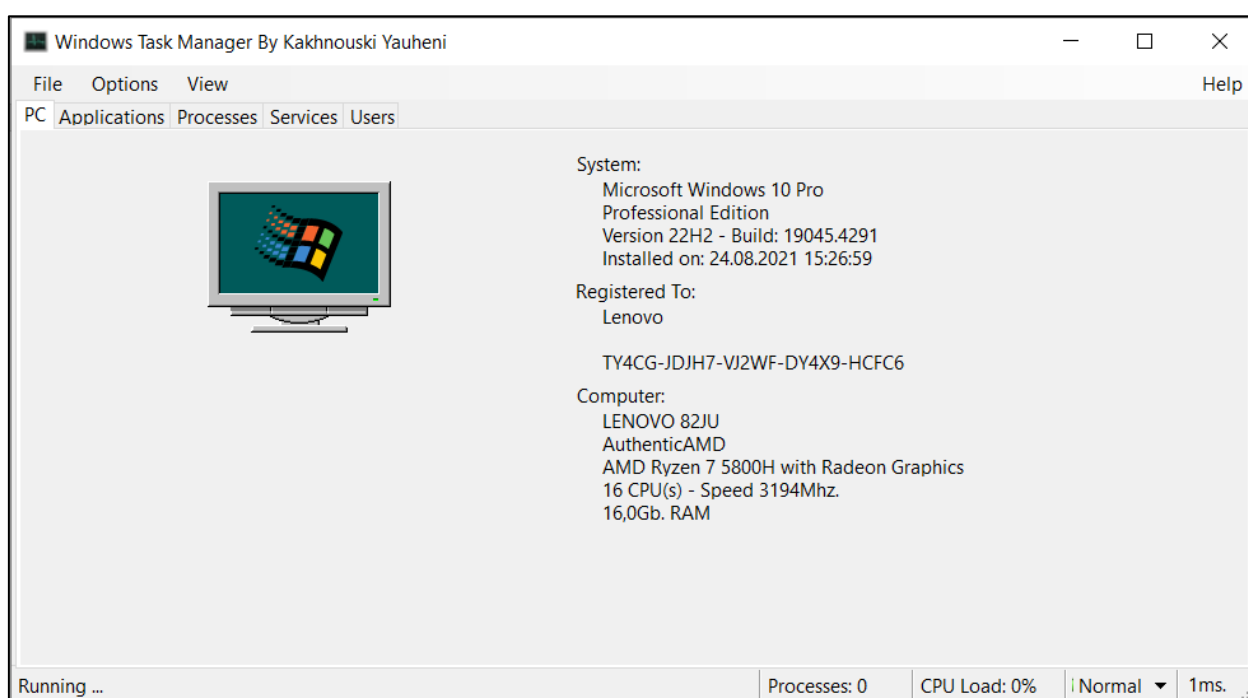


Рисунок 5.1 – Главное окно Task Manager

В статус баре можно увидеть три панели, такие как «File», «Options» и «View».

При нажатии на кнопку «File» появляется выпадающий список (рисунок 5.2) с возможностью выбрать одну из опций.

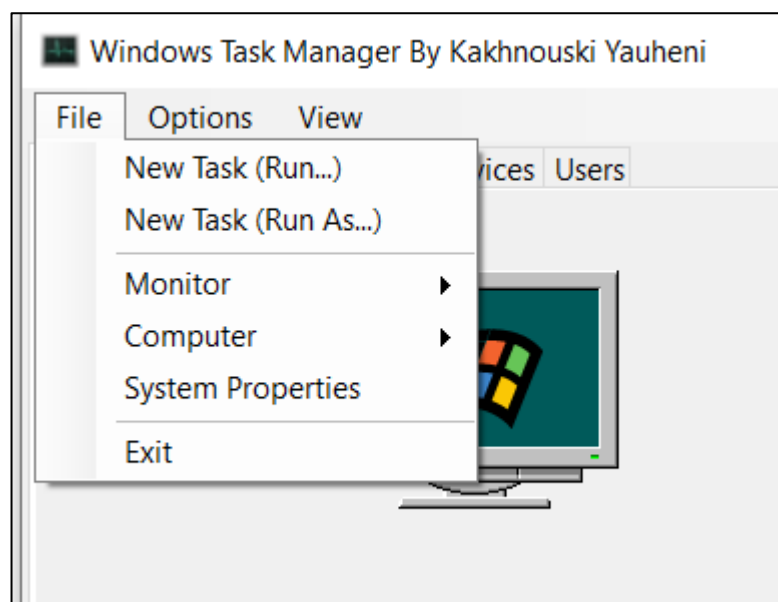


Рисунок 5.2 – Список опций «File»

Основные опции «File»:

- 1 Опция New Task и позволяет создать новый процесс в системе.
- 2 Опция Monitor открывает подписок, в котором есть функции скриншота экрана и выключение монитора.
- 3 Опция Computer открывает большой перечень дополнительных опций (рисунок 5.3), которые позволяют управлять состоянием компьютера. Можно перезагрузить, заблокировать, выключить, перевести в состояние гибернации систему.

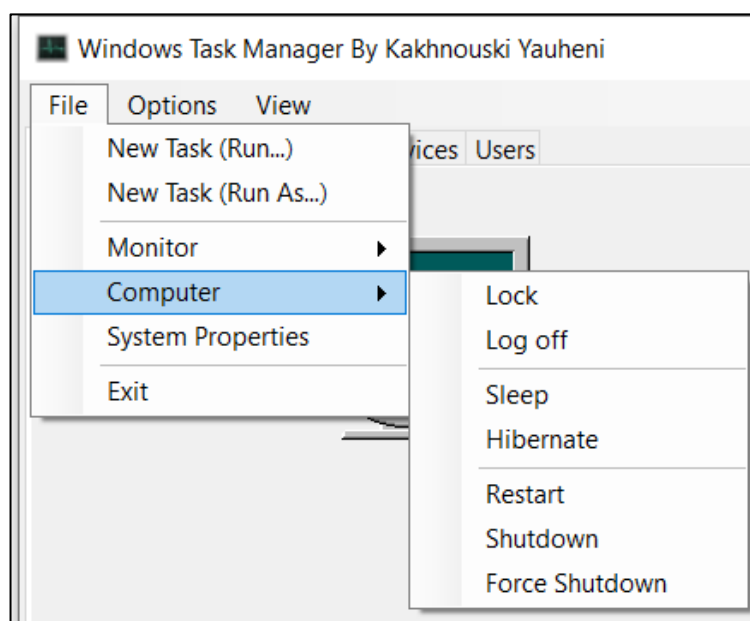


Рисунок 5.3 – Список опций «Computer»

4 Опция System Properties выводит дополнительное окно с характеристиками персонального компьютера.

5 Опция Exit завершает работу приложения.

При нажатии на кнопку «Options» появляется выпадающий список (рисунок 5.4) с возможностью выбрать одну из опций.

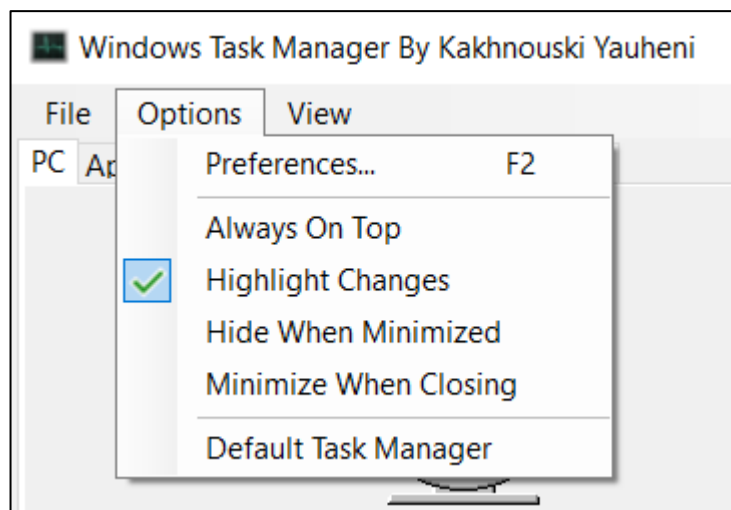


Рисунок 5.4 – Список опций «Options»

Основные опции «Options»:

1 Опция Preferences позволяет настроить палитру цветов для событий изменения, фиксирования, добавления и удаление элементов Task Manager (рисунок 5.5).

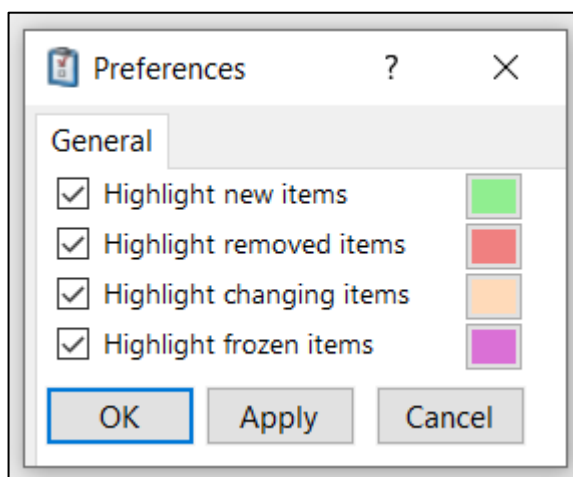


Рисунок 5.5 – Окно «Preferences»

2 Опция Always on Top закрепляет иконку Task Manager на панели задач.

3 Опция Default Task Manager заменяет в реестре системы вызов

стандартного диспетчера задач на Task Manager.

При нажатии на кнопку «View» появляется выпадающий список (рисунок 5.6) с возможностью выбрать одну из опций.

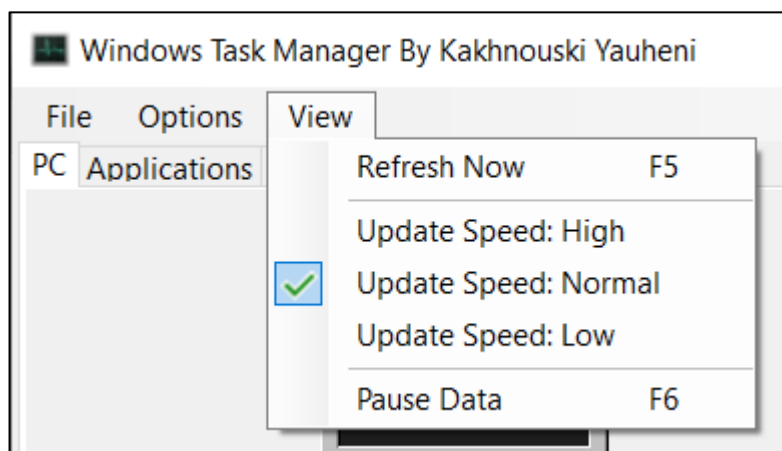


Рисунок 5.6 – Список опций «View»

Основные опции «View» взаимодействуют с частотой обновления данных в Task Manager. Можно обновить данные вручную, можно регулировать скорость обновления данных, а можно поставить обновление на паузу, тем самым диспетчер задач остановит свое выполнение, пока обновление не возобновится.

Вкладка «Help», позволяет вывести информацию о приложении (рисунок 5.7) и задержку обновления данных (рисунок 5.8).

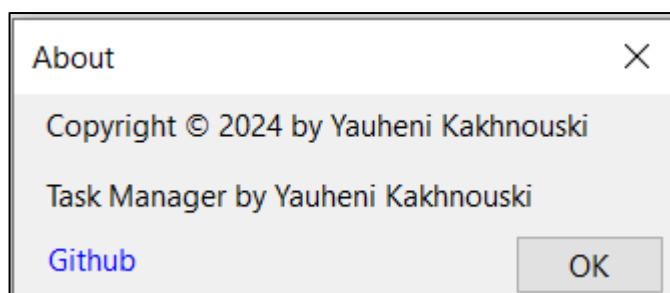


Рисунок 5.7 – Окно «About»

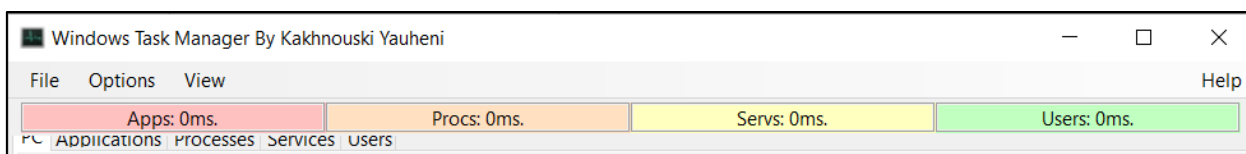


Рисунок 5.8 – Вывод задержек обновления данных

В нижнем сайд-баре можно увидеть дополнительную информацию о загрузке центрального процессора, о скорости обновления данных и о количестве запущенных процессов (рисунок 5.9).

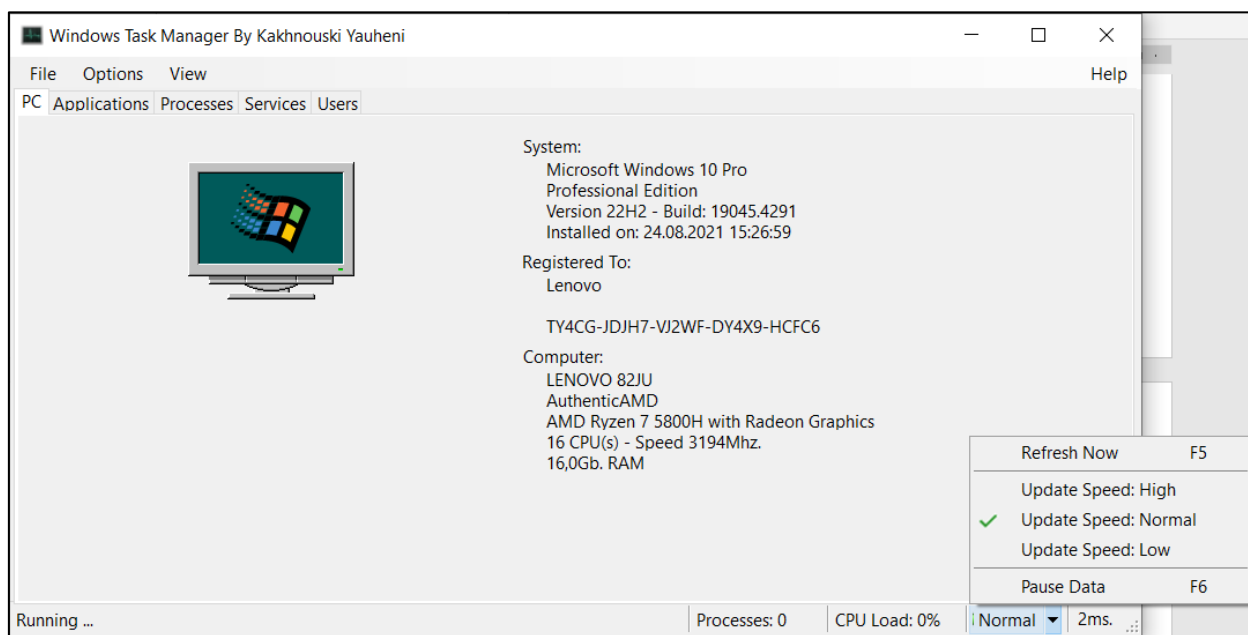


Рисунок 5.9 – Информация на нижнем сайд-баре

5.2 Приложения

Task Manager предоставляет подробную информацию о запущенных приложениях, их названиях, PID, Handle и Status.

Все это находится на вкладке «Applications» (рисунок 5.10).

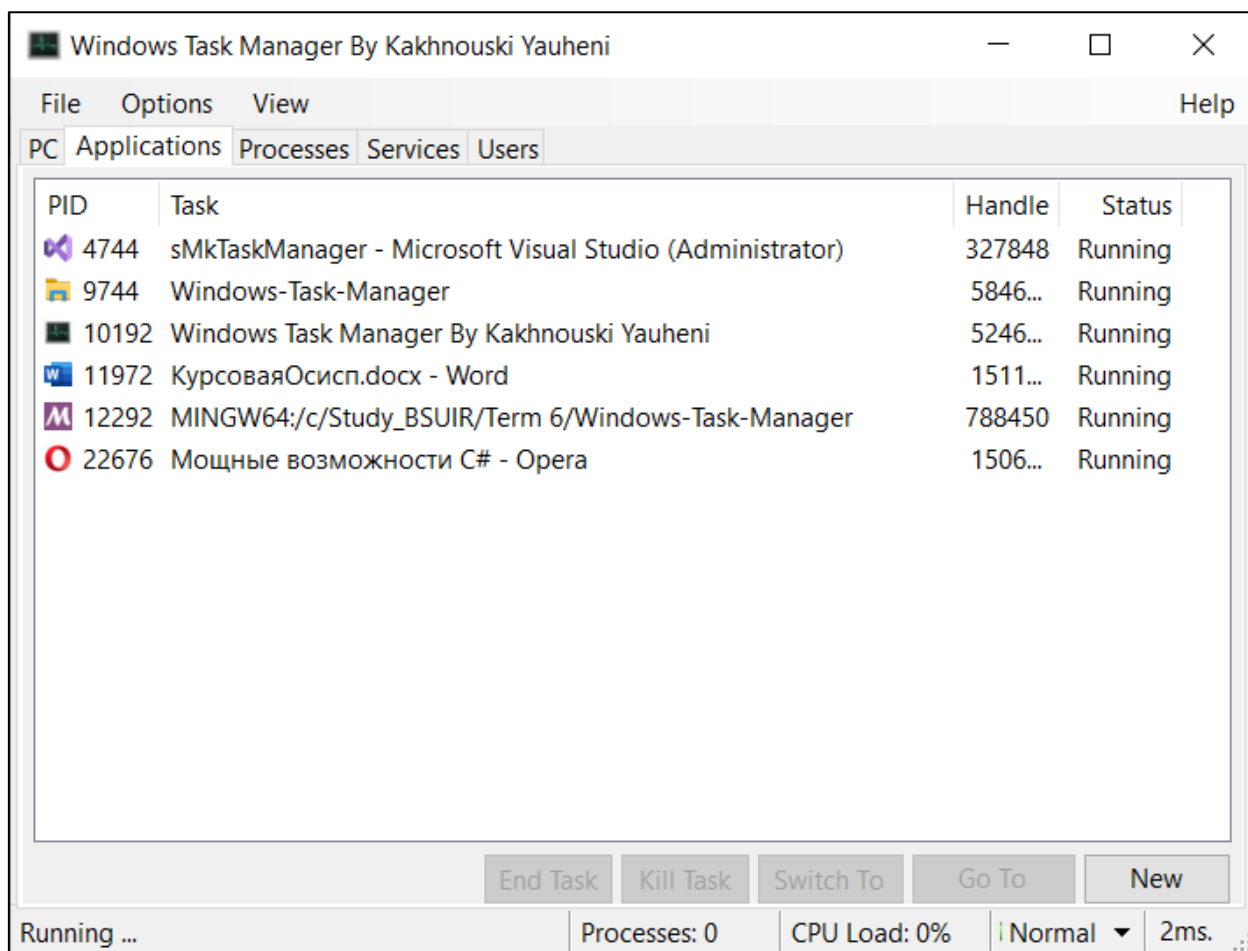


Рисунок 5.10 – Вкладка «Applications»

При нажатии правой кнопкой мыши на любое приложение, открывается выпадающий список с возможностями управления этим приложением (рисунок 5.11).

В этом списке присутствуют такие опции как, перейти в каталог выполнения приложения, взаимодействие с потреблением ресурсов приложения, можно их минимизировать или увеличить, так же есть возможность завершить приложение, убить приложение и перейти к процессам, которые созданы в рамках данного приложения.

Почти всю ту же функциональность можно увидеть в нижнем сайд-баре. Из новых возможностей, там добавляется кнопка добавления нового

приложения.

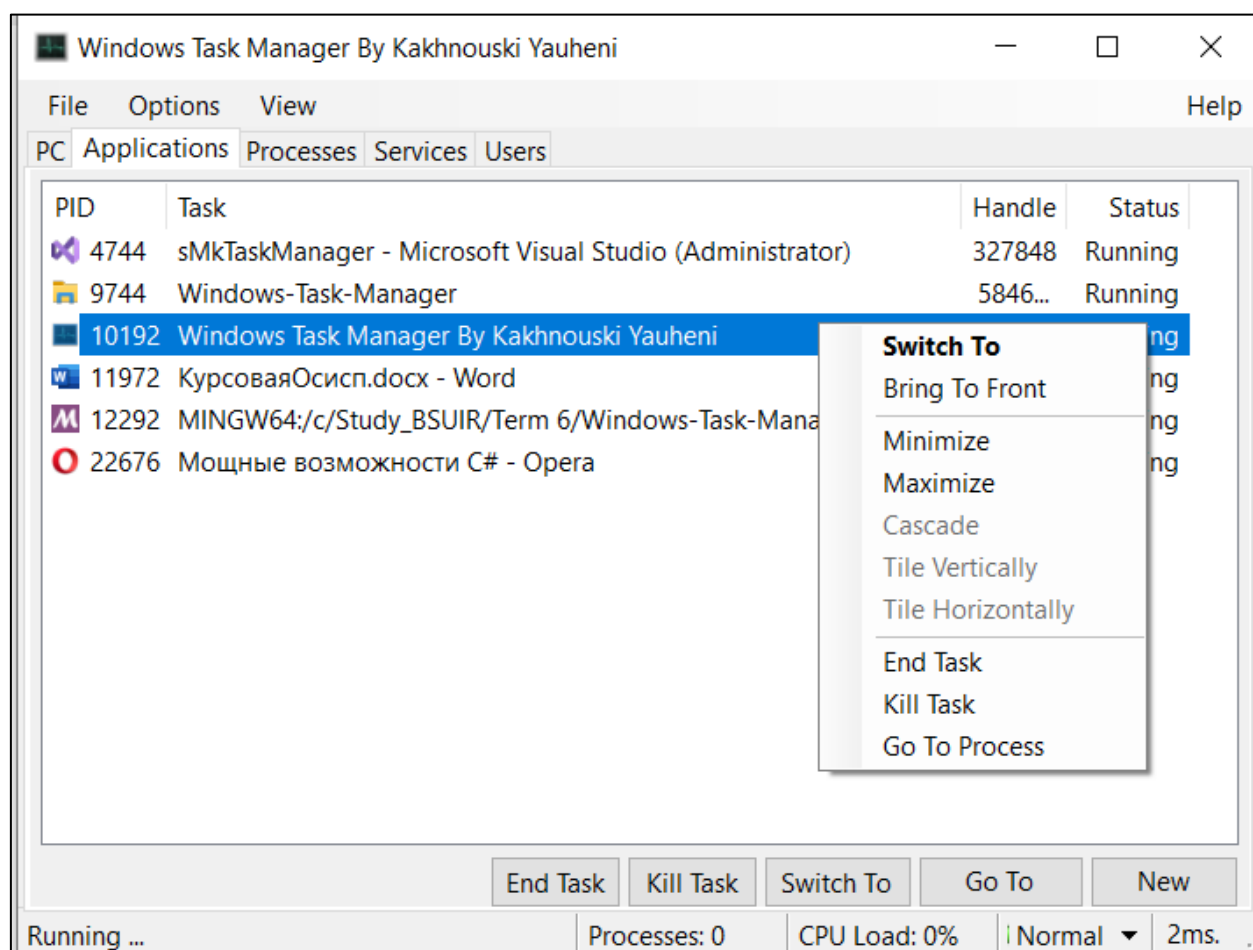


Рисунок 5.11 – Список опций выбранного приложения

5.3 Процессы

Task Manager предоставляет подробную информацию о запущенных в системе процессах, их названиях, описаниях, приоритете, PID и многое другое. Все это находится на вкладке «Processes» (рисунок 5.12).

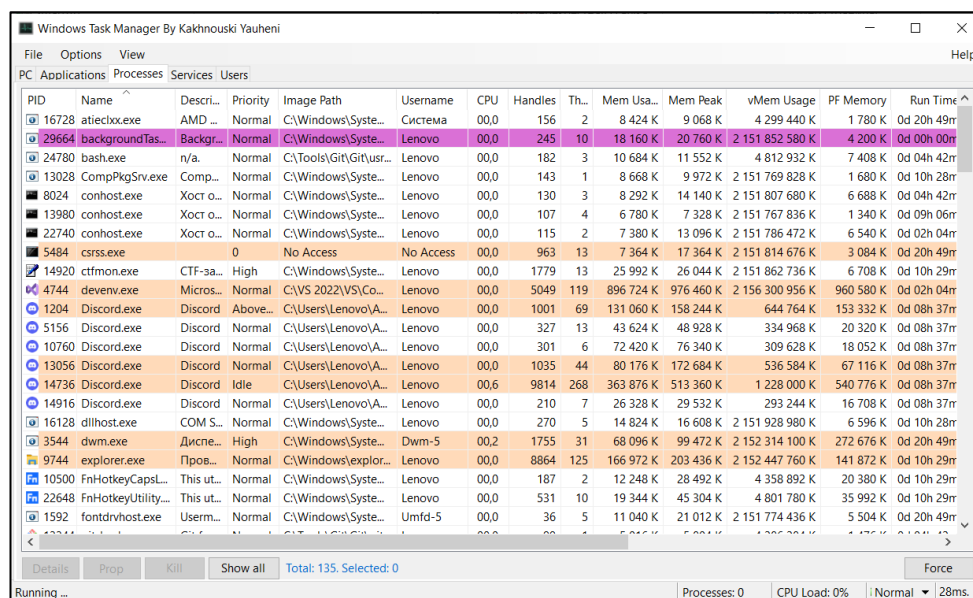


Рисунок 5.12 – Вкладка «Processes»

При нажатии правой кнопкой мыши на любой из запущенных процессов выпадет список с дополнительными возможностями, помимо стандартных опций управления процессом (завершение, обновление и тд.), так же есть опция вывода расширенной информации о процессе (рисунок 5.13).

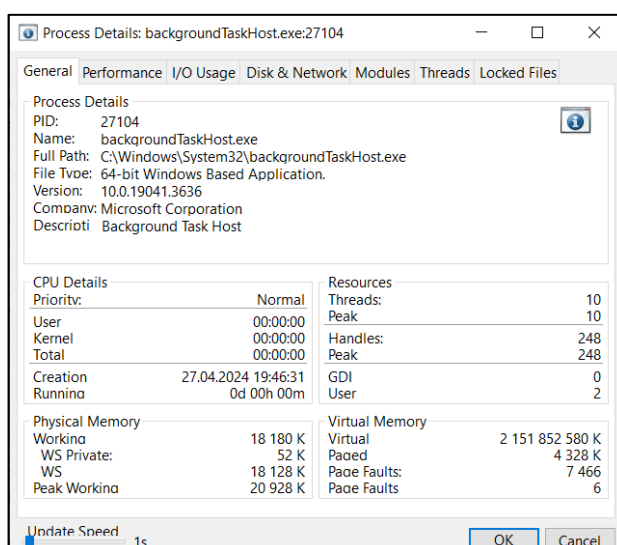


Рисунок 5.13 – Окно с расширенной информацией о процессе

5.4 Службы и пользователи

Task Manager предоставляет подробную информацию о запущенных в системе службах и пользователях, их названиях, описаниях, статусе, PID и многое другое.

Все это находится на вкладке «Services» (рисунок 5.14).

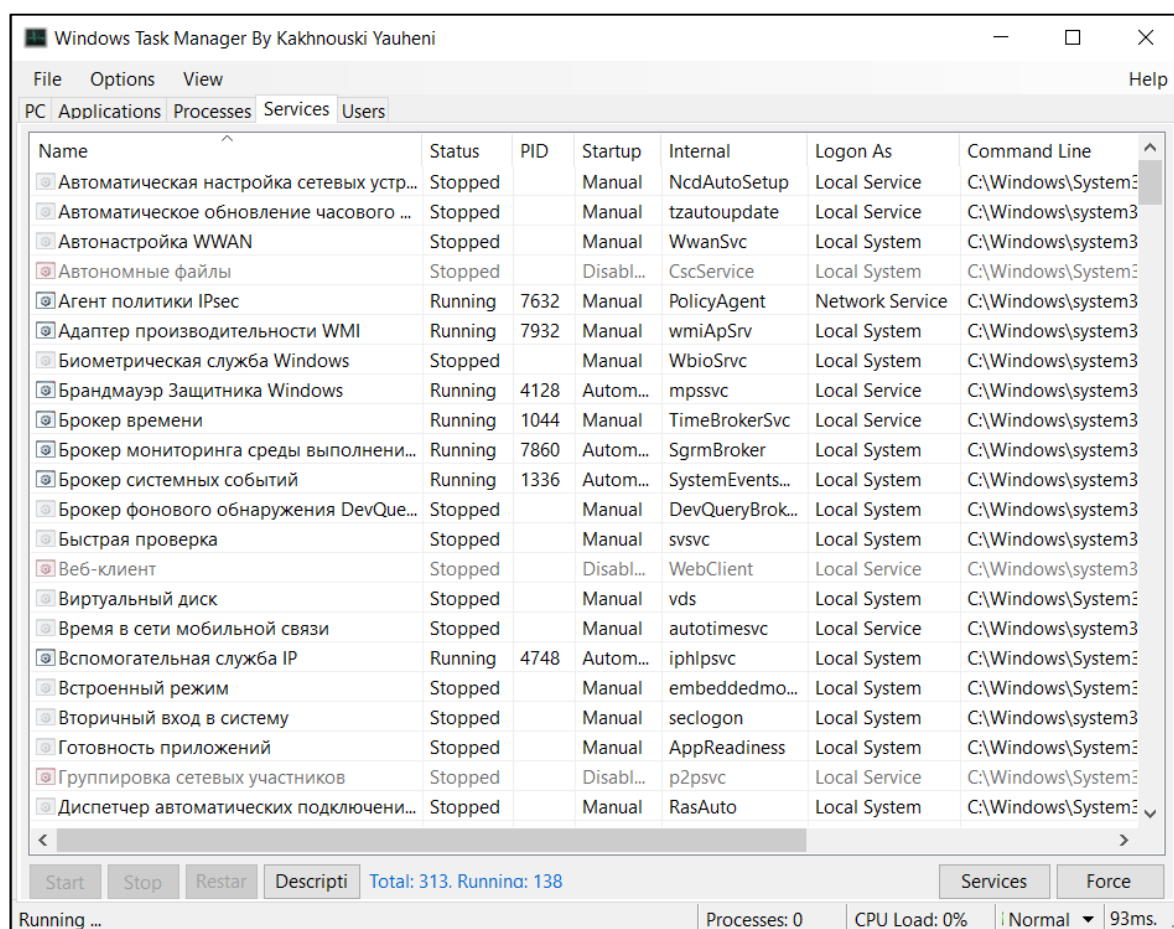


Рисунок 5.14 – Вкладка «Services»

При нажатии правой кнопкой мыши на любую из запущенных служб выпадет список с дополнительными возможностями.

Помимо стандартных опций управления службами (завершение, обновление, переход к процессам службы, открытие файла службы и тд.), так же есть опция вывода расширенной информации о службе (рисунок 5.15).

В нижнем сайд-баре можно увидеть различные кнопки для взаимодействия с состоянием службы, так же кнопку вызова стандартного окна служб Windows и кнопку принудительного завершения работы службы.

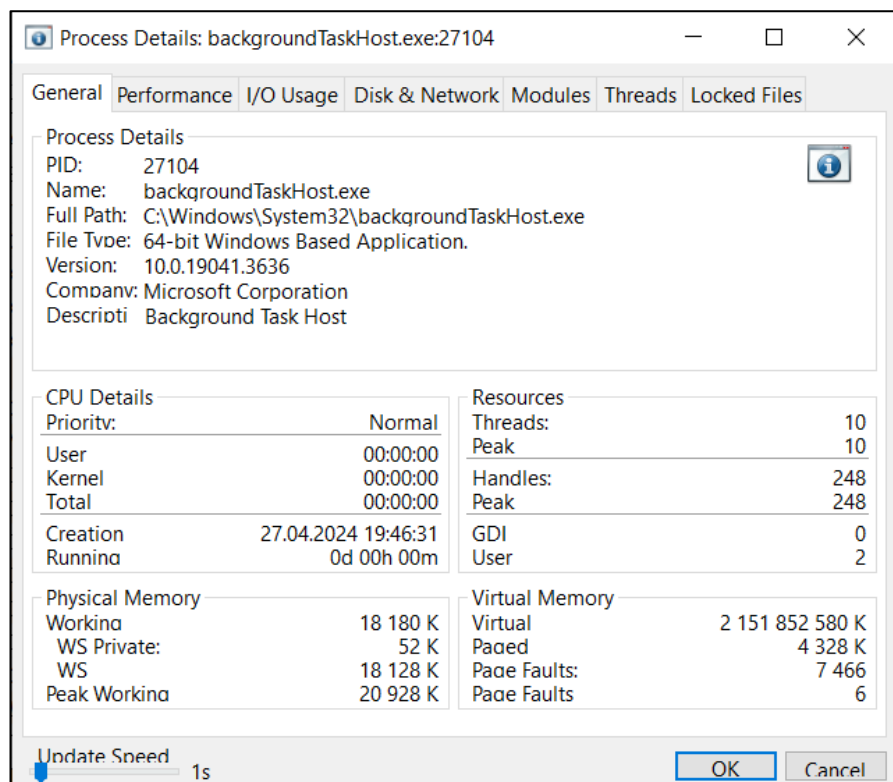


Рисунок 5.15 – Окно с расширенной информацией о службе

Информация о активных пользователях доступна на вкладке «Users» (рисунок 5.16).

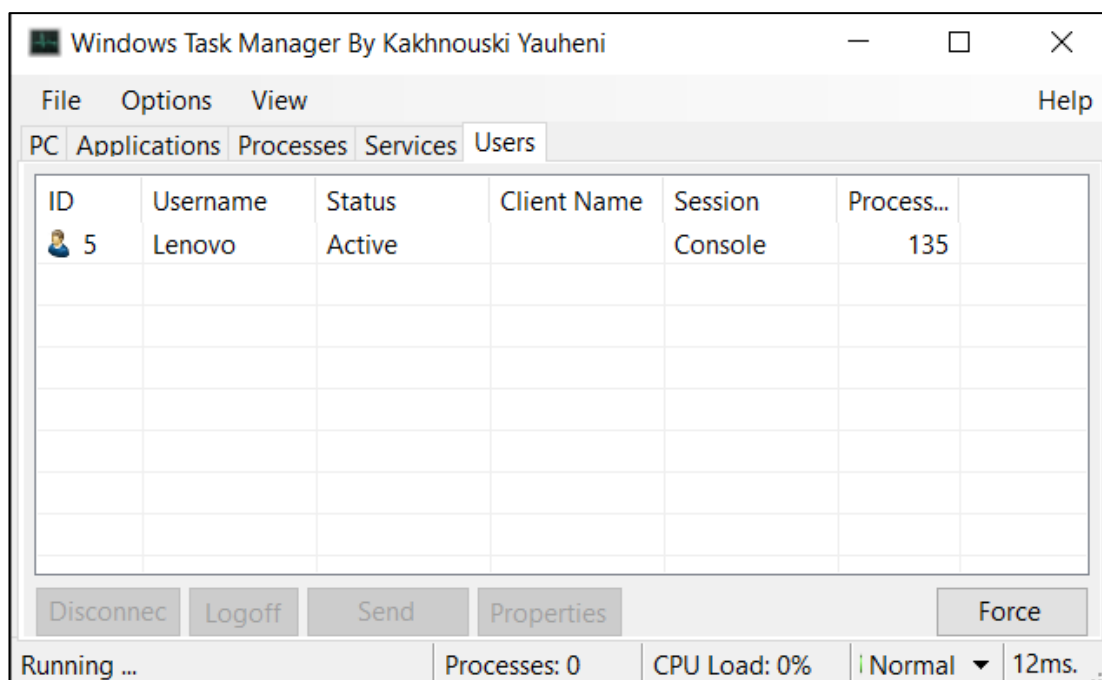


Рисунок 5.16 – Вкладка «Users»

ЗАКЛЮЧЕНИЕ

В рамках курсового проекта было проведено исследование и разработка аналога Task Manager для операционной системы Windows.

Основной задачей проекта было создание инструмента, предназначенного для мониторинга текущих процессов, приложений, служб, анализа их использования ресурсов и управления ими с целью оптимизации работы и предоставления пользователю более широких возможностей контроля системы.

Для достижения этой цели были рассмотрены основные аспекты управления процессами, приложениями, службами и пользователями в операционной системе Windows.

Особое внимание было уделено диспетчеру задач Windows, который представляет собой мощный инструмент для мониторинга и управления процессами, приложениями и службами.

Анализ основных функциональных возможностей диспетчера задач позволил понять его роль в контроле над ресурсами компьютера, оптимизации работы системы и выявлении потенциальных проблемных ситуаций.

Изучение принципов работы диспетчера задач с процессами, приложениями и службами позволило сделать вывод о его важности для обеспечения стабильной и эффективной работы операционной системы Windows.

Так же внимание было уделено разработке эффективных алгоритмов мониторинга и управления процессами. Для этого использовался язык программирования C# и фреймворк WinForms для создания удобного и интуитивно понятного пользовательского интерфейса. Технология WMI (Windows Management Instrumentation) была выбрана для получения информации о процессах и ресурсах системы, что позволило эффективно взаимодействовать с операционной системой и предоставить пользователю всю необходимую информацию.

В результате работы был разработан аналог Task Manager для Windows, который предоставляет пользователям возможность более эффективного контроля за работой операционной системы.

Созданное приложение позволяет оптимизировать процессы, приложения, службы в соответствии с потребностями и требованиями пользователей, что ведет к улучшению производительности и стабильности работы операционной системы Windows.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

- [1] Доманов, А. Т. Стандарт предприятия / А. Т. Доманов, Н. И. Сорока. – Минск: БГУИР, 2017. – 167 с.
- [2] Рихтер, Windows для профессионалов: создание эффективных Win32-приложений с учетом специфики 64разрядной версии Windows / Рихтер. – СПб: Питер; М.: Издательско-торговый дом "Русская редакция", 2003. – 654 с.
- [3] История развития Microsoft Windows [Электронный ресурс]. – Режим доступа: <https://tproger.ru/translations/microsoft-windows-history>. – Дата доступа: 20.02.2024.
- [4] Кларк, Программирование серверных приложений для Microsoft Windows 2000 / Кларк, Рихтер. – СПб: Питер; М.: Издательско-торговый дом "Русская редакция", 2001. – 507 с.
- [5] Кастер Хелен, Основы Windows NT и NTFS / Кастер Хелен, Рихтер. – Москва: М.: Русская редакция., 1996. – 361 с.
- [6] Карпов, В.Е. Основы операционных систем / В.Е. Карпов, К. А. Коньков. – Москва: Издательство "Интуит.ру", 2005 г.– 2-е издании. – 486 с.
- [7] Русинович, М. Внутреннее устройство Microsoft Windows: Windows Server 2003, Windows XP и Windows 2000 / М. Русинович, Д. Соломон. – СПб.: Питер : М.: Издательско-торговый дом "Русская редакция";, 2005. – 347 с.
- [8] Столлинкс, В. Операционные системы / В. Столлинкс, Д. Соломон. – СПб.: Питер: М.: Вильямс, 2001. – 715 с.
- [9] Харт, Д. Системное программирование в среде Win32 / Д. Харт, Д. Соломон. – СПб.: Питер : М.: Издательский дом "Вильямс", 2001. – 258 с.
- [10] Таненбаум, Э. Современные операционные системы / Э. Таненбаум, Д. Соломон. – СПб.: Питер: СПб.: Издательский дом Питер, 2002. – 1034 с.
- [11] Ховард, Разработка защищенных Web-приложений на платформе Microsoft Windows 2000 / Ховард, Д. – СПб.: Питер: М.: Издательско-торговый дом "Русская редакция", 2001. – 252 с.
- [12] Матвеев, М.Д. Самоучитель Microsoft Windows XP: всё об использовании и настройках / М.Д. Матвеев – СПб.: Питер: М.: Издательско-торговый дом "Русская редакция", 2005. – 620 с.
- [13] Свиридова, М.Ю. Самоучитель Microsoft Windows XP: Все об использовании и настройках : учеб. пособие для нач. проф. образования / М.Ю. Свиридова – СПб.: Питер : -М.: Академия, 2006. – 189 с.

[14] Гордеев, А.В. Операционные системы: учеб, для вузов по напр. «Информатика и вычисл. техн»/ А.В. Гордеев – СПб.: Питер : -М.: Академия, 2004. – 415 с.

[15] Большаков, Т.В. Операционные системы: учеб. пособие/Д.В. Иртегов; / Т.В. Большаков – Новосибирск: НГУ.- Новосибирск:, 2005. – 136 с.

[16] Леонтьев, В.П. Осваиваем Windows XP быстро и увлекательно: справочное издание/ В.П. Леонтьев. – : - М. : ОЛМА-ПРЕСС, 2005. – 219 с.

[17] Сергеенко, А.С. Windows 98: Справочник / А.С. Сергеенко, Д. Соломон. – –СПб: Питер : - М. : ОЛМА-ПРЕСС, 1999. – 448 с.

[18] Бойс, Дж. Осваиваем Windows 95: Пер. с англ. /Под общей ред. Лейко И.П. / Дж. Бойс – –СПб: Питер : –М.: Бином, 1995. – 368 с.

[19] Дейтел, Х. М. Операционные системы. Том 1. Основы и принципы / Х. М. Дейтел. – –СПб: Питер : –М.: Бином, 2019. – 498 с.

[20] Русинович, М. Внутреннее устройство Microsoft Windows. Основные подсистемы ОС / М. Русинович. – –СПб: Питер : - М.: Питер, 2015. – 486 с.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

Листинг А.1 – ApisHandles.cs

```
using System.Text;
using System.Diagnostics;
using System.Runtime.InteropServices;
using System.Runtime.Versioning;
namespace sMkTaskManager.Classes;

[SupportedOSPlatform("windows")]
internal unsafe static partial class Handles {
    const int MAX_PATH = 260;
    const int DUPLICATE_SAME_ACCESS = 0x2;
    const int CNST_SYSTEM_HANDLE_INFORMATION = 16;
    const uint STATUS_INFO_LENGTH_MISMATCH = 0xC0000004U;

    public static List<string> GetFilesLockedBy(Process process) {
        try {
            var retFiles = new List<string>();
            foreach (SYSTEM_HANDLE_INFORMATION h in GetHandles(process)) {
                try {
                    var File = GetFilePath(h, process);
                    if (File != null) {
                        retFiles.Add(File);
                    }
                } catch (Exception ex) { Shared.DebugTrap(ex, 1901); }
            }
            return retFiles;
        } catch (Exception ex) { Shared.DebugTrap(ex, 1902); return new List<string>(); }
    }

    public static List<SYSTEM_HANDLE_INFORMATION> GetHandles(Process process)
    {
        uint nStatus = 0;
        int nHandleInfoSize = 0x10000;
        IntPtr ipHandlePointer = Marshal.AllocHGlobal(nHandleInfoSize);
        int nLength = 0;

        while (InlineAssignHelper(ref nStatus,
            NtQuerySystemInformation(CNST_SYSTEM_HANDLE_INFORMATION, ipHandlePointer,
            nHandleInfoSize, ref nLength)) == STATUS_INFO_LENGTH_MISMATCH) {
            nHandleInfoSize = nLength;
            Marshal.FreeHGlobal(ipHandlePointer);
            ipHandlePointer = Marshal.AllocHGlobal(nLength);
        }

        long lHandleCount;
        IntPtr ipHandle;
        if (Shared.Is64Bits()) {
            lHandleCount = Marshal.ReadInt64(ipHandlePointer);
        }
    }
}
```

```

        ipHandle = new IntPtr(ipHandlePointer.ToInt64() + 8);
    } else {
        lHandleCount = Marshal.ReadInt32(ipHandlePointer);
        ipHandle = new IntPtr(ipHandlePointer.ToInt32() + 4);
    }

    List<SYSTEM_HANDLE_INFORMATION> lstHandles = new();
    for (long lIndex = 0; lIndex < lHandleCount; lIndex++) {
        SYSTEM_HANDLE_INFORMATION? shHandle = new();
        if (Shared.Is64Bits()) {
            shHandle =
            (SYSTEM_HANDLE_INFORMATION?)Marshal.PtrToStructure(ipHandle,
            shHandle.GetType());
            ipHandle = new IntPtr(ipHandle.ToInt64() +
            Marshal.SizeOf(shHandle) + 8);
        } else {
            ipHandle = new IntPtr(ipHandle.ToInt64() +
            Marshal.SizeOf(shHandle));
            shHandle =
            (SYSTEM_HANDLE_INFORMATION?)Marshal.PtrToStructure(ipHandle,
            shHandle.GetType());
        }
        if (shHandle == null) continue;
        if (shHandle?.ProcessID != process.Id) continue;
        lstHandles.Add(shHandle.Value);
    }
    Marshal.FreeHGlobal(ipHandlePointer);
    return lstHandles;
}

public static string? GetFilePath(SYSTEM_HANDLE_INFORMATION
SYSTEM_HANDLE_INFORMATION, Process process) {
    if (SYSTEM_HANDLE_INFORMATION.GrantedAccess == 0x12019F ||
SYSTEM_HANDLE_INFORMATION.GrantedAccess == 0x1A019F ||
SYSTEM_HANDLE_INFORMATION.GrantedAccess == 0x120189 ||
SYSTEM_HANDLE_INFORMATION.GrantedAccess == 0x100000) {
        return null;
    }

    IntPtr m_ipProcessHwnd = OpenProcess(ProcessAccessFlags.All, false,
process.Id);
    IntPtr ipHandle = IntPtr.Zero;
    var objBasic = new OBJECT_BASIC_INFORMATION();
    IntPtr ipBasic;
    var objObjectType = new OBJECT_TYPE_INFORMATION();
    IntPtr ipObjectType;
    var objObjectName = new OBJECT_NAME_INFORMATION();
    IntPtr ipObjectName;
    string strObjectName = "";
    int nLength = 0;
    int nReturn = 0;
    IntPtr ipTemp;

    if (!DuplicateHandle(m_ipProcessHwnd,
SYSTEM_HANDLE_INFORMATION.Handle, GetCurrentProcess(), ref ipHandle, 0, false,

```

```

    DUPLICATE_SAME_ACCESS)) {
        return null;
    }

    ipBasic = Marshal.AllocHGlobal(Marshal.SizeOf(objBasic));
    _ = NtQueryObject(ipHandle,
    Convert.ToInt32(ObjectInformationClass.ObjectBasicInformation), ipBasic,
    Marshal.SizeOf(objBasic), ref nLength);
    objBasic = (OBJECT_BASIC_INFORMATION)Marshal.PtrToStructure(ipBasic,
    objBasic.GetType());
    Marshal.FreeHGlobal(ipBasic);

    ipObjectType = Marshal.AllocHGlobal(objBasic.TypeInformationLength);
    nLength = objBasic.TypeInformationLength;
    while (Convert.ToUInt32(InlineAssignHelper(ref nReturn,
    NtQueryObject(ipHandle,
    Convert.ToInt32(ObjectInformationClass.ObjectTypeInfoInformation), ipObjectType,
    nLength, ref nLength))) == STATUS_INFO_LENGTH_MISMATCH) {
        Marshal.FreeHGlobal(ipObjectType);
        ipObjectType = Marshal.AllocHGlobal(nLength);
    }

    objObjectType =
    (OBJECT_TYPE_INFORMATION)Marshal.PtrToStructure(ipObjectType,
    objObjectType.GetType());
    if (Shared.Is64Bits()) {
        ipTemp = new
        IntPtr(Convert.ToInt64(objObjectType.Name.Buffer.ToString(), 10) >> 32);
    } else {
        ipTemp = objObjectType.Name.Buffer;
    }

    string strObjectName = Marshal.PtrToStringUni(ipTemp,
    objObjectType.Name.Length >> 1);
    Marshal.FreeHGlobal(ipObjectType);
    if (strObjectName != "File") { return null; }

    // nLength = objBasic.NameInformationLength
    nLength = 0x1000;
    ipObjectName = Marshal.AllocHGlobal(nLength);
    while (Convert.ToUInt32(InlineAssignHelper(ref nReturn,
    NtQueryObject(ipHandle,
    Convert.ToInt32(ObjectInformationClass.ObjectNameInformation), ipObjectName,
    nLength, ref nLength))) == STATUS_INFO_LENGTH_MISMATCH) {
        Marshal.FreeHGlobal(ipObjectName);
        ipObjectName = Marshal.AllocHGlobal(nLength);
    }
    objObjectName =
    (OBJECT_NAME_INFORMATION)Marshal.PtrToStructure(ipObjectName,
    objObjectName.GetType());

    if (Shared.Is64Bits()) {
        ipTemp = new
        IntPtr(Convert.ToInt64(objObjectName.Name.Buffer.ToString(), 10) >> 32);
    }

```

```

    } else {
        ipTemp = objObjectName.Name.Buffer;
    }
    Marshal.FreeHGlobal(ipObjectName);

    if (ipTemp != IntPtr.Zero) {
        byte[] baTemp = new byte[nLength];
        try {
            Marshal.Copy(ipTemp, baTemp, 0, nLength);
            strObjectName = Marshal.PtrToStringUni(Shared.Is64Bits() ? new
IntPtr(ipTemp.ToInt64()) : new IntPtr(ipTemp.ToInt32()))!;
        } catch (AccessViolationException) {
            return null;
        } finally {
            _ = CloseHandle(ipHandle);
        }
    }

    return GetRegularFileNameFromDevice(strObjectName);

}

public static T InlineAssignHelper<T>(ref T target, T value) {
    target = value;
    return value;
}

public static string GetRegularFileNameFromDevice(string strRawName) {
    string strFileName = strRawName;
    foreach (string strDrivePath in Environment.GetLogicalDrives()) {
        StringBuilder sbTargetPath = new(MAX_PATH);
        if (QueryDosDevice(strDrivePath[..2], sbTargetPath, MAX_PATH) == 0)
        {
            return strRawName;
        }
        string strTargetPath = sbTargetPath.ToString();
        if (strFileName.StartsWith(strTargetPath)) {
            strFileName = strFileName.Replace(strTargetPath,
strDrivePath[..2]);
            break;
        }
    }
    return strFileName;
}

public static string GetRegularKeyName(string strInternalKey) {
    strInternalKey =
strInternalKey.Replace(@"\REGISTRY\MACHINE\SYSTEM\CURRENTCONTROLSET\HARDW
ARE PROFILES\CURRENT", "HKCC");
    strInternalKey =
strInternalKey.Replace(@"\REGISTRY\MACHINE\SOFTWARE\CLASSES", "HKCR");
    strInternalKey = strInternalKey.Replace(@"\REGISTRY\USER\S",
"HKU\S");
    strInternalKey = strInternalKey.Replace(@"\REGISTRY\MACHINE",
"HKLM");
    strInternalKey = strInternalKey.Replace(@"\REGISTRY\USER", "HKCU");
    return strInternalKey;
}

```

```

    }

    public enum ObjectInformationClass : int {
        ObjectBasicInformation = 0,
        ObjectNameInformation = 1,
        ObjectTypeInformation = 2,
        ObjectAllTypesInformation = 3,
        ObjectHandleInformation = 4
    }

    [Flags()] public enum ProcessAccessFlags : uint {
        All = 0x1F0FFF,
        Terminate = 0x1,
        CreateThread = 0x2,
        VMOperation = 0x8,
        VMRead = 0x10,
        VMWrite = 0x20,
        DupHandle = 0x40,
        SetInformation = 0x200,
        QueryInformation = 0x400,
        Synchronize = 0x100000
    }

    [StructLayout(LayoutKind.Sequential)]
    OBJECT_BASIC_INFORMATION {
        public int Attributes;
        public int GrantedAccess;
        public int HandleCount;
        public int PointerCount;
        public int PagedPoolUsage;
        public int NonPagedPoolUsage;
        public int Reserved1;
        public int Reserved2;
        public int Reserved3;
        public int NameInformationLength;
        public int TypeInformationLength;
        public int SecurityDescriptorLength;
        public System.Runtime.InteropServices.ComTypes.FILETIME CreateTime;
    }

    [StructLayout(LayoutKind.Sequential)]
    OBJECT_TYPE_INFORMATION {
        public UNICODE_STRING Name;
        public int TotalNumberOfObjects;
        public int TotalNumberOfHandles;
        public int TotalPagedPoolUsage;
        public int TotalNonPagedPoolUsage;
        public int TotalNamePoolUsage;
        public int TotalHandleTableUsage;
        public int HighWaterNumberOfObjects;
        public int HighWaterNumberOfHandles;
        public int HighWaterPagedPoolUsage;
        public int HighWaterNonPagedPoolUsage;
        public int HighWaterNamePoolUsage;
        public int HighWaterHandleTableUsage;
        public int InvalidAttributes;
    }

```

```

        public GENERIC_MAPPING GenericMapping;
        public int ValidAccessMask;
        public bool SecurityRequired;
        public bool MaintainHandleCount;
        public int PoolType;
        public int DefaultPagedPoolCharge;
        public int DefaultNonPagedPoolCharge;
    }
    [StructLayout(LayoutKind.Sequential)] public struct
OBJECT_NAME_INFORMATION {
        public UNICODE_STRING Name;
    }
    [StructLayout(LayoutKind.Sequential, Pack = 1)] public struct
UNICODE_STRING {
        public ushort Length;
        public ushort MaximumLength;
        public IntPtr Buffer;
    }
    [StructLayout(LayoutKind.Sequential)] public struct GENERIC_MAPPING {
        public int GenericRead;
        public int GenericWrite;
        public int GenericExecute;
        public int GenericAll;
    }
    [StructLayout(LayoutKind.Sequential, Pack = 1)] public struct
SYSTEM_HANDLE_INFORMATION {
        public int ProcessID;
        public byte ObjectTypeNumber;
        public byte Flags;
        public ushort Handle;
        public int Object_Pointer;
        public uint GrantedAccess;
    }
}

```

```

[System.Security.SuppressUnmanagedCodeSecurity()] [DllImport("ntdll.dll",
CharSet = CharSet.Unicode, SetLastError = true)]
private static extern int NtQueryObject(IntPtr ObjectHandle, int
ObjectInformationClass, IntPtr ObjectInformation, int ObjectInformationLength,
ref int returnLength);
[System.Security.SuppressUnmanagedCodeSecurity()] [DllImport("ntdll.dll",
CharSet = CharSet.Unicode, SetLastError = true)]
private static extern uint NtQuerySystemInformation(int
SystemInformationClass, IntPtr SystemInformation, int SystemInformationLength,
ref int returnLength);
[System.Security.SuppressUnmanagedCodeSecurity()]
[DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
private static extern uint QueryDosDevice(string lpDeviceName,
StringBuilder lpTargetPath, int ucchMax);
[System.Security.SuppressUnmanagedCodeSecurity()]
[DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
private static extern IntPtr GetCurrentProcess();
[System.Security.SuppressUnmanagedCodeSecurity()]
[DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
private static extern IntPtr OpenProcess(ProcessAccessFlags

```

```

dwDesiredAccess, [MarshalAs(UnmanagedType.Bool)] bool bInheritHandle, int
dwProcessId);
[System.Security.SuppressUnmanagedCodeSecurity()]
[DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
private static extern bool DuplicateHandle(IntPtr hSourceProcessHandle,
ushort hSourceHandle, IntPtr hTargetProcessHandle, ref IntPtr lpTargetHandle,
uint dwDesiredAccess, [MarshalAs(UnmanagedType.Bool)] bool bInheritHandle, uint
dwOptions);
[System.Security.SuppressUnmanagedCodeSecurity()]
[DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
private static extern int CloseHandle(IntPtr hObject);

}

```

Листинг A.2 – TaskManagerBase.cs

```

internal class TaskManagerValuesBase {

    public class TaskManagerMetricChangedEventArgs {
        public TaskManagerMetricChangedEventArgs(Metric metric,
MetricChangedEventArgs e) {
            this.metric = metric;
            this.e = e;
        }
        public Metric metric { get; set; }
        public MetricChangedEventArgs e { get; set; }
    }

    public TaskManagerValuesBase() {
        // Attach the event handlers for all the Metrics discovered
        var properties = GetType().GetProperties().Where(p => p.PropertyType
== typeof(Metric));
        foreach (var p in properties) {
            var metric = (Metric)p.GetValue(this)!;
            metric.AnyChanged += OnMetricChanged;
            metric.ValueChanged += OnMetricValueChanged;
            metric.DeltaChanged += OnMetricDeltaChanged;
        }
    }

    protected void OnMetricChanged(Metric sender, MetricChangedEventArgs e) {
if (!CancellingEvents) MetricChanged?.Invoke(this, sender, e); }
    protected void OnMetricValueChanged(Metric sender, MetricChangedEventArgs
e) { if (!CancellingEvents) MetricValueChanged?.Invoke(this, sender, e); }
    protected void OnMetricDeltaChanged(Metric sender, MetricChangedEventArgs
e) { if (!CancellingEvents) MetricDeltaChanged?.Invoke(this, sender, e); }
    public event MetricEventHandler? MetricChanged;
    public event MetricEventHandler? MetricValueChanged;
    public event MetricEventHandler? MetricDeltaChanged;
    public delegate void MetricEventHandler(object sender, Metric metric,
MetricChangedEventArgs e);

    public void SetValue(Metric m, Int128 newValue) => m.SetValue(newValue);
    public bool CancellingEvents = false;
    public long LastUpdate = 0;
}

```

Листинг А.3 – TaskManagerProcess.cs

```
using System.ComponentModel;
using System.Diagnostics;
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;
using System.Runtime.Versioning;
using System.Security.Principal;
using System.Text;

namespace sMkTaskManager.Classes;

[SupportedOSPlatform("windows")]
internal class TaskManagerProcess : IEquatable<TaskManagerProcess>,
INotifyPropertyChanged {
    private readonly int _PID;
    private IntPtr _pHandle = IntPtr.Zero;

    public TaskManagerProcess(int pid) {
        _PID = pid;
        LastUpdated = DateTime.Now.Ticks;
        LastChanged = LastUpdated;
        PreviousUpdate = LastUpdated;
        PropertyChanged += MyPropertyChanged;
    }
    public TaskManagerProcess(IntPtr pid) : this(pid.ToInt32()) { }

    public string ID => _PID.ToString();
    public int PID => _PID;
    private Color _BackColor;
    public Color BackColor { get => _BackColor; set { SetField(ref
_BackColor, value); } }
    public long LastUpdated { get; set; }
    public long LastChanged { get; set; }
    public long PreviousUpdate { get; set; } = 0;
    public bool NotifyChanges { get; set; } = true;
    public int ImageIndex { get; set; } = -1;
    public string ImageKey { get; set; } = "";
    public bool IgnoreBackColor { get; set; } = false;
    public Icon? Icon { get; private set; } = null;

    private bool _Suspended;
    public bool Suspended { get => _Suspended; set { SetField(ref _Suspended,
value); BackColor = value ? Settings.Highlights.FrozenColor : Color.Empty; }
}

    /* Primary Basic Properties */
    private string _Name = "", _Description = "", _ImagePath = "",
_CommandLine = "";
    private string _Affinity = "", _Priority = "", _Username = "", _RunTime =
"", _CpuUsage = "";
    private uint _SessionID, _Handles, _Threads; private ulong _PageFaults;
    private int _GDIObjects, _UserObjects, _GDIObjectsPeak, _UserObjectsPeak;
    private IntPtr _ParentPID;
    public string Name { get => _Name; set { SetField(ref _Name, value); } }
    public IntPtr ParentPID { get => _ParentPID; set { SetField(ref
_ParentPID, value); } }
    public string Description { get => _Description; set { SetField(ref
_Description, value); } }
    public string ImagePath { get => _ImagePath; set { SetField(ref
_ImagePath, value); } }
    public string CommandLine { get => _CommandLine; set { SetField(ref
```

```

        public GENERIC_MAPPING GenericMapping;
        public int ValidAccessMask;
        public bool SecurityRequired;
        public bool MaintainHandleCount;
        public int PoolType;
        public int DefaultPagedPoolCharge;
        public int DefaultNonPagedPoolCharge;
    }
    [StructLayout(LayoutKind.Sequential)] public struct
OBJECT_NAME_INFORMATION {
        public UNICODE_STRING Name;
    }
    [StructLayout(LayoutKind.Sequential, Pack = 1)] public struct
UNICODE_STRING {
        public ushort Length;
        public ushort MaximumLength;
        public IntPtr Buffer;
    }
    [StructLayout(LayoutKind.Sequential)] public struct GENERIC_MAPPING {
        public int GenericRead;
        public int GenericWrite;
        public int GenericExecute;
        public int GenericAll;
    }
    [StructLayout(LayoutKind.Sequential, Pack = 1)] public struct
SYSTEM_HANDLE_INFORMATION {
        public int ProcessID;
        public byte ObjectTypeNumber;
        public byte Flags;
        public ushort Handle;
        public int Object_Pointer;
        public uint GrantedAccess;
    }

    [System.Security.SuppressUnmanagedCodeSecurity()] [DllImport("ntdll.dll",
CharSet = CharSet.Unicode, SetLastError = true)]
    private static extern int NtQueryObject(IntPtr ObjectHandle, int
ObjectInformationClass, IntPtr ObjectInformation, int ObjectInformationLength,
ref int returnLength);
    [System.Security.SuppressUnmanagedCodeSecurity()] [DllImport("ntdll.dll",
CharSet = CharSet.Unicode, SetLastError = true)]
    private static extern uint NtQuerySystemInformation(int
SystemInformationClass, IntPtr SystemInformation, int SystemInformationLength,
ref int returnLength);
    [System.Security.SuppressUnmanagedCodeSecurity()]
    [DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
    private static extern uint QueryDosDevice(string lpDeviceName,
StringBuilder lpTargetPath, int ucchMax);
    [System.Security.SuppressUnmanagedCodeSecurity()]
    [DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]
    private static extern IntPtr GetCurrentProcess();
    [System.Security.SuppressUnmanagedCodeSecurity()]
    [DllImport("kernel32.dll", CharSet = CharSet.Unicode, SetLastError = true)]

```


ПРИЛОЖЕНИЕ Б
(обязательное)
Функциональная схема

ПРИЛОЖЕНИЕ В
(обязательное)
Блок схема алгоритма

ПРИЛОЖЕНИЕ Г
(обязательное)
Скриншоты работы программы

ПРИЛОЖЕНИЕ Д
(обязательное)
Ведомость документов